

SOLIDWORKS API (.NET VBA C#) — Examples (15/50):

Create_Reference_Curve_Example_CSharp-

Create_Sketch_Point_Example_VB

SOLIDWORKS API Documentation (converted from CHM)

Create Reference Curve Example (C#)

This example shows how to create a reference curve by first creating a temporary spline curve.

```
//-----
// Preconditions:
// 1. Verify that the specified part document template exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens a new part document.
// 2. Adds a reference curve to the part document.
// 3. Selects an endpoint on the reference curve
//    and prints to the Immediate window the
//    endpoint's position and coordinates.
// 4. Examine the graphics area and Immediate window.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace EdgePointCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            PartDoc swPart = default(PartDoc);
            Body2 swBody = default(Body2);
            Curve[] swCurve = new Curve[1];
            object vCurve = null;
            ReferenceCurve swRefCurve = default(ReferenceCurve);
            object swSelObj = null;
            SelectionMgr swSelMgr = default(SelectionMgr);
            EdgePoint swEdgePoint = default(EdgePoint);
            bool bRet = false;
            int nRetVal = 0;
            int iDim = 0;
            int iOrd = 0;
            int incp = 0;
            int iper = 0;
            double[] dprops = new double[2];
            double[] knots = new double[10];
            double[] cPoints = new double[18];
            object vprops = null;
            object vknots = null;
            object vcPoints = null;
            int selType = 0;
            double x = 0;
            double y = 0;
            double z = 0;
            int iValueIn1 = 65535;
            int iValueIn2 = 345;
            double dValueOut = 0;
            DoubleIntConv.Pack(iValueIn1, iValueIn2, ref dValueOut);

            //Open new part document and create a body
            swModel = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SolidWorks\\SolidWorks 2015\\templates\\Part.prtdot", 0, 0, 0);
            swPart = (PartDoc)swModel;
            swBody = (Body2)swPart.CreateNewBody();

            //Create a curve
            //Set properties
            iDim = 3;
            iOrd = 4;
            incp = 6;
            iper = 0;
            DoubleIntConv.Pack(iDim, iOrd, ref dprops[0]);
            DoubleIntConv.Pack(incp, iper, ref dprops[1]);
            vprops = dprops;

            //Set knots
            knots[0] = 0;
            knots[1] = 0;
            knots[2] = 0;
            knots[3] = 0;
            knots[4] = 0.33096;
            knots[5] = 0.72;
            knots[6] = 1;
            knots[7] = 1;
            knots[8] = 1;
            knots[9] = 1;
            vknots = knots;

            //Set control points
            cPoints[0] = 0;
            cPoints[1] = 0;
            cPoints[2] = 0;
            cPoints[3] = 0.008703;
            cPoints[4] = 0.016501;
            cPoints[5] = 0;
            cPoints[6] = 0.027636;
            cPoints[7] = 0.052399;
            cPoints[8] = 0;
            cPoints[9] = 0.069472;
            cPoints[10] = -0.011297;
            cPoints[11] = 0;
            cPoints[12] = 0.090421;
            cPoints[13] = 0.017622;
            cPoints[14] = 0;
            cPoints[15] = 0.099188;
            cPoints[16] = 0.029725;
            cPoints[17] = 0;
            vcPoints = cPoints;
        }
    }
}
```

```

//Create a spline curve
swCurve[0] = (Curve)swBody.AddProfileBspline((vprops), (vknots), (vcPoints));
vCurve = (object)swCurve;

//Create a reference curve
swRefCurve = (ReferenceCurve)swModel.FeatureReferenceCurve(1, (vCurve), true, "", out nRetVal);

//Rebuild to display curve
swModel.EditRebuild3();

//Get endpoint on reference curve
swModelDocExt = (ModelDocExtension)swModel.Extension;
bRet = swModelDocExt.SelectByID("Unknown", "POINTREF", 0, 0, 0, false, 0, null, 0);
swSelMgr = (SelectionMgr)swModel.SelectionManager;
selType = swSelMgr.GetSelectedObjectType3(1, -1);
Debug.Print("Type of selection: " + selType);
if ((int)swSelectType_e.swSelPOINTREFS == selType)
{
    swSelObj = (object)swSelMgr.GetSelectedObject6(1, -1);
    swEdgePoint = (EdgePoint)swSelObj;
    Debug.Print(" Position of the endpoint: " + swEdgePoint.Position);
    swEdgePoint.GetPointCoordinates(out x, out y, out z);
    Debug.Print(" Endpoint coordinates: " + x + ", " + y + ", and " + z);
}
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}

[System.Runtime.InteropServices.StructLayout(LayoutKind.Explicit)]
public class DoubleIntConv
{
    //An 8-byte double contains 2 4-byte integers
    [System.Runtime.InteropServices.FieldOffset(0)]
    private int m_Int1;
    [System.Runtime.InteropServices.FieldOffset(4)]
    private int m_Int2;
    [System.Runtime.InteropServices.FieldOffset(0)]
    private double m_Double;

    private DoubleIntConv(double dValue)
    {
        //C# wants these initialized in the constructor
        m_Int1 = 0;
        m_Int2 = 0;
        m_Double = dValue;
    }

    private DoubleIntConv(int iValue1, int iValue2)
    {
        //C# wants these initialized in the constructor
        m_Double = 0.0;
        m_Int1 = iValue1;
        m_Int2 = iValue2;
    }

    // Use an out parameter, so client code can pass in an uninitialized variable
    public static void Pack(int iIn1, int iIn2, ref double dOut)
    {
        DoubleIntConv cv = null;
        cv = new DoubleIntConv(iIn1, iIn2);
        dOut = cv.m_Double;
    }
}
}
}

```

Create Reference Curve Example (VBA)

This example shows how to create a reference curve by first creating a temporary spline curve.

```
'-----
' Preconditions:
' 1. Verify that the specified part document template exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens a new part document.
' 2. Adds a reference curve to the part document.
' 3. Selects an endpoint on the reference curve
'    and prints to the Immediate window the
'    endpoint's position and coordinates.
' 4. Examine the graphics area and Immediate window.
'-----

Option Explicit

'Type definitions
Type DoubleRec
    dValue As Double
End Type

Type Int2Rec
    iLower As Long        '1 Integer is 4 bytes
    iUpper As Long
End Type

'Pack 2 integers to 1 Double
Function ImportFields(iLower As Integer, iUpper As Integer, dValue As Double)
    Dim dr As DoubleRec
    Dim i2r As Int2Rec

    i2r.iLower = iLower
    i2r.iUpper = iUpper
    LSet dr = i2r
    dValue = dr.dValue
End Function

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swModelDocExt As SldWorks.ModelDocExtension
    Dim swPart As SldWorks.PartDoc
    Dim swBody As SldWorks.Body2
    Dim swCurve(0) As SldWorks.Curve
    Dim vCurve As Variant
    Dim swRefCurve As SldWorks.ReferenceCurve
    Dim swSelObj As Object
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swEdgePoint As SldWorks.EdgePoint
    Dim bRet As Boolean
    Dim nRetVal As Long
    Dim iDim As Integer
    Dim iOrd As Integer
    Dim incp As Integer
    Dim iper As Integer
    Dim dprops(1) As Double
    Dim knots(9) As Double
    Dim cPoints(17) As Double
    Dim vprops As Variant
    Dim vknots As Variant
    Dim vcPoints As Variant
    Dim selType As Long
    Dim x As Double
    Dim y As Double
    Dim z As Double

    'Open new part document and create a body
    Set swApp = Application.SldWorks
    Set swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SolidWorks 2015\templates\Part.prtdot", 0, 0, 0)
    Set swPart = swModel
    Set swBody = swPart.CreateNewBody

    'Create a curve
    'Set properties
    iDim = 3: iOrd = 4: incp = 6: iper = 0
    ImportFields iDim, iOrd, dprops(0)
    ImportFields incp, iper, dprops(1)
    vprops = dprops

    'Set knots
    knots(0) = 0: knots(1) = 0: knots(2) = 0: knots(3) = 0
    knots(4) = 0.33096: knots(5) = 0.72
    knots(6) = 1: knots(7) = 1: knots(8) = 1: knots(9) = 1
    vknots = knots

    'Set control points
    cPoints(0) = 0: cPoints(1) = 0: cPoints(2) = 0
    cPoints(3) = 0.008703: cPoints(4) = 0.016501: cPoints(5) = 0
    cPoints(6) = 0.027636: cPoints(7) = 0.052399: cPoints(8) = 0
    cPoints(9) = 0.069472: cPoints(10) = -0.011297: cPoints(11) = 0
```

```

cPoints(12) = 0.090421: cPoints(13) = 0.017622: cPoints(14) = 0
cPoints(15) = 0.099188: cPoints(16) = 0.029725: cPoints(17) = 0
vcPoints = cPoints

'Create a spline curve
Set swCurve(0) = swBody.AddProfileBspline((vprops), (vknots), (vcPoints))
vCurve = swCurve

'Create a reference curve
Set swRefCurve = swModel.FeatureReferenceCurve(1, (vCurve), True, "", nRetVal)

'Rebuild to display curve
swModel.EditRebuild3

'Get endpoint on reference curve
Set swModelDocExt = swModel.Extension
bRet = swModelDocExt.SelectByID2("Unknown", "POINTREF", 0, 0, 0, False, 0, Nothing, 0)
Set swSelMgr = swModel.SelectionManager
selType = swSelMgr.GetSelectedObjectType3(1, -1)
Debug.Print "Type of selection: " & selType
If swSelectType_e.swSelPOINTREFS = selType Then
    Set swSelObj = swSelMgr.GetSelectedObject6(1, -1)
    Set swEdgePoint = swSelObj
    Debug.Print " Position of the endpoint: " & swEdgePoint.Position
    swEdgePoint.GetPointCoordinates x, y, z
    Debug.Print " Endpoint coordinates: " & x & ", " & y & ", and " & z
End If

End Sub

```

Create Reference Curve Example (VB.NET)

This example shows how to create a reference curve by first creating a temporary spline curve.

```

'-----
' Preconditions:
' 1. Verify that the specified part document template exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens a new part document.
' 2. Adds a reference curve to the part document.
' 3. Selects an endpoint on the reference curve
'    and prints to the Immediate window the
'    endpoint's position and coordinates.
' 4. Examine the graphics area and Immediate window.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swPart As PartDoc
        Dim swBody As Body2
        Dim swCurve(0) As Curve
        Dim vCurve As Object
        Dim swRefCurve As ReferenceCurve
        Dim swSelObj As Object
        Dim swSelMgr As SelectionMgr
        Dim swEdgePoint As EdgePoint
        Dim bRet As Boolean
        Dim nRetVal As Integer
        Dim iDim As Integer
        Dim iOrd As Integer
        Dim incp As Integer
        Dim iper As Integer
        Dim dprops(1) As Double
        Dim knots(9) As Double
        Dim cPoints(17) As Double
        Dim vprops As Object
        Dim vknots As Object
        Dim vcPoints As Object
        Dim selType As Integer
        Dim x As Double
        Dim y As Double
        Dim z As Double
        Dim iValueIn1 As Integer = 65535
        Dim iValueIn2 As Integer = 345
        Dim dValueOut As Double
        DoubleIntConv.Pack(iValueIn1, iValueIn2, dValueOut)

        'Open new part document and create a body
        swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SolidWorks 2015\templates\Part.prtdot", 0, 0, 0)
        swPart = swModel
        swBody = swPart.CreateNewBody

        'Create a curve
        'Set properties
        iDim = 3 : iOrd = 4 : incp = 6 : iper = 0
        DoubleIntConv.Pack(iDim, iOrd, dprops(0))
        DoubleIntConv.Pack(incp, iper, dprops(1))
        vprops = dprops

        'Set knots
        knots(0) = 0 : knots(1) = 0 : knots(2) = 0 : knots(3) = 0
        knots(4) = 0.33096 : knots(5) = 0.72
        knots(6) = 1 : knots(7) = 1 : knots(8) = 1 : knots(9) = 1
        vknots = knots

        'Set control points
        cPoints(0) = 0 : cPoints(1) = 0 : cPoints(2) = 0
        cPoints(3) = 0.008703 : cPoints(4) = 0.016501 : cPoints(5) = 0
        cPoints(6) = 0.027636 : cPoints(7) = 0.052399 : cPoints(8) = 0
        cPoints(9) = 0.069472 : cPoints(10) = -0.011297 : cPoints(11) = 0
        cPoints(12) = 0.090421 : cPoints(13) = 0.017622 : cPoints(14) = 0
        cPoints(15) = 0.099188 : cPoints(16) = 0.029725 : cPoints(17) = 0
        vcPoints = cPoints

        'Create a spline curve
        swCurve(0) = swBody.AddProfileBspline((vprops), (vknots), (vcPoints))
        vCurve = swCurve

        'Create a reference curve
        swRefCurve = swModel.FeatureReferenceCurve(1, (vCurve), True, "", nRetVal)

        'Rebuild to display curve
        swModel.EditRebuild3()

        'Get endpoint on reference curve
        swModelDocExt = swModel.Extension
        bRet = swModelDocExt.SelectByID2("Unknown", "POINTREF", 0, 0, 0, False, 0, Nothing, 0)
        swSelMgr = swModel.SelectionManager
        selType = swSelMgr.GetSelectedObjectType3(1, -1)
        Debug.Print("Type of selection: " & selType)
        If swSelectType_e.swSelPOINTREFS = selType Then
            swSelObj = swSelMgr.GetSelectedObject6(1, -1)
            swEdgePoint = swSelObj
            Debug.Print("Position of the endpoint: " & swEdgePoint.Position)
            swEdgePoint.GetPointCoordinates(x, y, z)
            Debug.Print("Endpoint coordinates: " & x & ", " & y & ", and " & z)
        End If

    End Sub

End Sub

```

```

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class

<System.Runtime.InteropServices.StructLayout(LayoutKind.Explicit)> Public Class DoubleIntConv

    'An 8-byte double contains 2 4-byte integers
    <System.Runtime.InteropServices.FieldOffset(0)> Private m_Int1 As Integer
    <System.Runtime.InteropServices.FieldOffset(4)> Private m_Int2 As Integer
    <System.Runtime.InteropServices.FieldOffset(0)> Private m_Double As Double

    Private Sub New(ByVal dValue As Double)
        'VB.NET wants these initialized in the constructor
        m_Int1 = 0
        m_Int2 = 0
        m_Double = dValue
    End Sub

    Private Sub New(ByVal iValue1 As Integer, ByVal iValue2 As Integer)
        'VB.NET wants these initialized in the constructor
        m_Double = 0.0
        m_Int1 = iValue1
        m_Int2 = iValue2
    End Sub

    ' Use an out parameter, so client code can pass in an uninitialized variable
    Public Shared Sub Pack(ByVal iIn1 As Integer, ByVal iIn2 As Integer, ByRef dOut As Double)
        Dim cv As DoubleIntConv
        cv = New DoubleIntConv(iIn1, iIn2)
        dOut = cv.m_Double
    End Sub

End Class

```

Create_Relative_Drawing_View_Example_CSharp (Topic)

SOLIDWORKS API Help

Create Relative Drawing View Example (C#)

This example shows how to create a relative drawing view.

```
// *****
// Preconditions:
// 1. Open public_documents\samples\tutorial\api\maingrip.sldprt.
// 2. Select File > Make Drawing from Part.
// 3. Run the macro.
//
// Postconditions:
// 1. Iterates through the drawing views
//    in the View Palette and drops
//    *Current drawing view in the drawing.
// 2. Activates the part.
// 3. Selects two faces for the relative drawing view.
// 4. Activates the drawing.
// 5. Creates and inserts a relative drawing
//    view using the selected faces.
//
// NOTE: Because the part document is used elsewhere, do not
// save any changes when closing it.
// *****

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;

namespace CreateRelativeViewCSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        DrawingDoc swDrawing;
        View swView;
        ModelDocExtension swModelDocExt;
        string fileName;
        bool status;
        int errors;
        int warnings;
        int numViews;
        object[] viewNames;
        string viewName;
        string viewPaletteName;
        int i;

        public void Main()
        {
            swDrawing = (DrawingDoc)swApp.ActiveDoc;

            // Get number of views on View Palette
            numViews = 0;
            viewNames = (object[])swDrawing.GetDrawingPaletteViewNames();

            // Iterate through views on View Palette
            // When view name equals *Current, drop
            // that view in drawing
            if (!((viewNames == null)))
            {
                numViews = (viewNames.GetUpperBound(0) - viewNames.GetLowerBound(0));
                for (i = 0; i <= numViews; i++)
                {
                    viewPaletteName = (string)viewNames[i];
                    if ((viewPaletteName == "*Current"))
                    {
                        swView = (View)swDrawing.DropDrawingViewFromPalette2(viewPaletteName, 0.0, 0.0, 0.0, 0.0);
                    }
                }
            }

            // Activate the part document and
            // select two faces for the relative drawing view
            swApp.ActiveDoc3("maingrip.sldprt", false, (int)swRebuildOnActivation_e.swUserDecision, ref errors);
            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            swModel.ClearSelection2(true);
            status = swModelDocExt.SelectByID2("", "FACE", 0.0466263268498324, 0.005587999999987514, -0.00617351393179888, false, 1, null, 0);
            status = swModelDocExt.SelectByID2("", "FACE", 0.0504738910727269, 0.00167315253537481, -0.00496149996774875, true, 2, null, 0);

            // Activate the drawing document
            // Create and insert the relative drawing view using
            // the selected faces
            // Activate the relative drawing view
            swApp.ActivateDoc3("maingrip - Sheet1", false, (int)swRebuildOnActivation_e.swUserDecision, ref errors);
            swDrawing = (DrawingDoc)swApp.ActiveDoc;
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\maingrip.sldprt";
            swView = (View)swDrawing.CreateRelativeView(fileName, 0.203608914116486, 0.493530187561698, (int)swRelativeViewCreationDirection_e.swRelativeViewCreationDirection_FRONT, (int)swRelativeViewCreationDirection_e.swRelativeViewCreationDirection_RIGHT);
            status = swDrawing.ActivateView("Drawing View2");

        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>

        public SldWorks swApp;
    }
}
```

Create Relative Drawing View Example (VBA)

This example shows how to create a relative drawing view.

```
' *****
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\maingrip.sldprt.
' 2. Select File > Make Drawing from Part.
' 3. Run the macro.
'
' Postconditions:
' 1. Iterates through the drawing views
'    in the View Palette and drops
'    *Current drawing view in the drawing.
' 2. Activates the part.
' 3. Selects two faces for the relative drawing view.
' 4. Activates the drawing.
' 5. Creates and inserts a relative drawing
'    view using the selected faces.
'
' NOTE: Because the part document is used elsewhere, do not
' save any changes when closing it.
' *****
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swDrawing As SldWorks.DrawingDoc
Dim swView As SldWorks.View
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim fileName As String
Dim status As Boolean
Dim errors As Long
Dim warnings As Long
Dim numViews As Long
Dim viewNames As Variant
Dim viewName As Variant
Dim viewPaletteName As String

Sub main()

    Set swApp = Application.SldWorks
    Set swDrawing = swApp.ActiveDoc

    ' Get number of views on View Palette
    numViews = 0
    viewNames = swDrawing.GetDrawingPaletteViewNames

    ' Iterate through views on View Palette
    ' When view name equals *Current, drop
    ' that view in drawing
    If (Not (IsEmpty(viewNames))) Then
        numViews = (UBound(viewNames) - LBound(viewNames) + 1)
        For Each viewName In viewNames
            viewPaletteName = viewName
            If (viewPaletteName = "*Current") Then
                Set swView = swDrawing.DropDrawingViewFromPalette2(viewPaletteName, 0#, 0#, 0#)
            End If
        Next viewName
    End If

    ' Activate the part document and
    ' select two faces for the relative drawing view
    swApp.ActivateDoc3 "maingrip.sldprt", False, swRebuildOnActivation_e.swUserDecision, errors
    Set swModel = swApp.ActiveDoc
    Set swModelDocExt = swModel.Extension
    swModel.ClearSelection2 True
    status = swModelDocExt.SelectByID2("", "FACE", 4.66263268498324E-02, 5.58799999987514E-03, -6.17351393179888E-03, False, 1, Nothing, 0)
    status = swModelDocExt.SelectByID2("", "FACE", 5.04738910727269E-02, 1.67315253537481E-03, -4.96149996774875E-03, True, 2, Nothing, 0)

    ' Activate the drawing document
    ' Create and insert the relative drawing view using
    ' the selected faces
    ' Activate the relative drawing view
    swApp.ActivateDoc3 "maingrip - Sheet1", False, swRebuildOnActivation_e.swUserDecision, errors
    Set swDrawing = swApp.ActiveDoc
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\maingrip.sldprt"
    Set swView = swDrawing.CreateRelativeView(fileName, 0.203608914116486, 0.493530187561698, swRelativeViewCreationDirection_FRONT,
    swRelativeViewCreationDirection_RIGHT)
    status = swDrawing.ActivateView("Drawing View2")

End Sub
```

Create Relative Drawing View Example (VB.NET)

This example shows how to create a relative drawing view.

```
' *****
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\maingrip.sldprt.
' 2. Select File > Make Drawing from Part.
' 3. Run the macro.
'
' Postconditions:
' 1. Iterates through the drawing views
'    in the View Palette and drops
'    *Current drawing view in the drawing.
' 2. Activates the part.
' 3. Selects two faces for the relative drawing view.
' 4. Activates the drawing.
' 5. Creates and inserts a relative drawing
'    view using the selected faces.
'
' NOTE: Because the part document is used elsewhere, do not
' save any changes when closing it.
' *****
```

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
```

Partial Class SolidWorksMacro

```
Dim swModel As ModelDoc2
Dim swDrawing As DrawingDoc
Dim swView As View
Dim swModelDocExt As ModelDocExtension
Dim fileName As String
Dim status As Boolean
Dim errors As Integer
Dim warnings As Integer
Dim numViews As Integer
Dim numViews As Integer
Dim viewNames As Object
Dim viewName As Object
Dim viewPaletteName As String
```

Sub main()

```
swDrawing = swApp.ActiveDoc

' Get number of views on View Palette
numViews = 0
viewNames = swDrawing.GetDrawingPaletteViewNames

' Iterate through views on View Palette
' When view name equals *Current, drop
' that view in drawing
If (Not (IsNothing(viewNames))) Then
    numViews = (UBound(viewNames) - LBound(viewNames) + 1)
    For Each viewName In viewNames
        viewPaletteName = viewName
        If (viewPaletteName = "*Current") Then
            swView = swDrawing.DropDrawingViewFromPalette2(viewPaletteName, 0.0#, 0.0#, 0.0#)
        End If
    Next viewName
End If

' Activate the part document and
' select two faces for the relative drawing view
swApp.ActivateDoc3("maingrip.sldprt", False, swRebuildOnActivation_e.swUserDecision, errors)
swModel = swApp.ActiveDoc
swModelDocExt = swModel.Extension
swModel.ClearSelection2(True)
status = swModelDocExt.SelectByID2("", "FACE", 0.0466263268498324, 0.00558799999987514, -0.00617351393179888, False, 1, Nothing, 0)
status = swModelDocExt.SelectByID2("", "FACE", 0.0504738910727269, 0.00167315235337481, -0.00496149996774875, True, 2, Nothing, 0)

' Activate the drawing document
' Create and insert the relative drawing view using
' the selected faces
' Activate the relative drawing view
swApp.ActivateDoc3("maingrip - Sheet1", False, swRebuildOnActivation_e.swUserDecision, errors)
swDrawing = swApp.ActiveDoc
fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\maingrip.sldprt"
swView = swDrawing.CreateRelativeView(fileName, 0.203608914116486, 0.493530187561698, swRelativeViewCreationDirection_e.swRelativeViewCreationDirection_FRONT,
swRelativeViewCreationDirection_e.swRelativeViewCreationDirection_RIGHT
```

```
)
    status = swDrawing.ActivateView("Drawing View2")
```

End Sub

```
''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
```

End Class

Create Revolve Features Example (C#)

This example shows how to create revolve features.

```
//-----
// Preconditions:
// 1. Open a new part document.
// 2. Rename the namespace to match your C# project.
//
// Postconditions: Two revolve features and one cut-revolve feature are created.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
namespace FeatureRevolves_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        ModelDocExtension swModelDocExt;
        FeatureManager swFeatMgr;
        SelectionMgr swSelMgr;

        bool boolstatus;

        public void Main()
        {
            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelDocExt = swModel.Extension;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;

            // Create an axis
            boolstatus = swModelDocExt.SelectByID2("Right Plane", "PLANE", 0, 0, 0, true, 0, null,
(int)swSelectOption_e.swSelectOptionDefault);
            boolstatus = swModelDocExt.SelectByID2("Top Plane", "PLANE", 0, 0, 0, true, 0, null,
(int)swSelectOption_e.swSelectOptionDefault);
            swModel.InsertAxis2(true);

            // Create a rectangle
            boolstatus = swModelDocExt.SelectByID2("Top Plane", "PLANE", -0.08954836342753, 0.0004336873289503, 0.006720765739942,
false, 0, null, (int)swSelectOption_e.swSelectOptionDefault);
            swModel.InsertSketch2(true);
            swModel.ClearSelection2(true);
            swModel.SketchRectangle(-0.05668466821757, -0.02198379306525, 0, -0.01330857427717, 0.03972855876814, 0, true);

            // Create the first revolve feature
            swModel.InsertSketch2(true);
            swModel.ShowNamedView2("Trimetric", 8);

            boolstatus = swModelDocExt.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, false, 0, null,
(int)swSelectOption_e.swSelectOptionDefault);
            boolstatus = swModelDocExt.SelectByID2("Axis1", "AXIS", 0, 0, 0, true, 16, null,
(int)swSelectOption_e.swSelectOptionDefault);
            swFeatMgr = swModel.FeatureManager;

            swFeatMgr.FeatureRevolve2(true, true, false, false, false, false, 0, 0, 6.28318530718, 0, false,
false, 0.01, 0.01, 0, 0, 0, true, true, true);

            // Create a cut-revolve feature using a face on the revolve feature
            swSelMgr.EnableContourSelection = false;
            boolstatus = swModelDocExt.SelectByID2("", "FACE", -0.03095803920934, 0.01509536510872, 0.02198379306526, false, 0,
null, (int)swSelectOption_e.swSelectOptionDefault);
            swModel.InsertSketch2(true);
            swModel.ClearSelection2(true);
            swModel.SketchRectangle(-0.04194874421597, 0.01774859621099, 0, -0.01883036471929, -0.01265654504095, 0, true);
            swModel.InsertSketch2(true);

            boolstatus = swModelDocExt.SelectByID2("Sketch2", "SKETCH", 0, 0, 0, false, 0, null,
(int)swSelectOption_e.swSelectOptionDefault);
            boolstatus = swModelDocExt.SelectByID2("Line4@Sketch2", "EXTSKETCHSEGMENT", -0.01883036471929, 0.003802500010693, 0,
true, 4, null, (int)swSelectOption_e.swSelectOptionDefault);
            swFeatMgr.FeatureRevolveCut(6.26573201466, false, 0, 0, 0, true, true);

            // Create the second revolve feature using a face on the first revolve feature
            swSelMgr.EnableContourSelection = false;
            boolstatus = swModelDocExt.SelectByID2("", "FACE", -0.02333512246603, 0.03472018719853, 0.0219837930652, false, 0,
null, (int)swSelectOption_e.swSelectOptionDefault);
            swModel.InsertSketch2(true);
            swModel.ClearSelection2(true);
            swModel.CreateCircle2(-0.02232361399104, 0.03354683337932, 0, -0.01445073476016, 0.02795861773112, 0);
            swModel.InsertSketch2(true);

            boolstatus = swModelDocExt.SelectByID2("Sketch3", "SKETCH", 0, 0, 0, false, 0, null,
(int)swSelectOption_e.swSelectOptionDefault);
            boolstatus = swModelDocExt.SelectByRay(-1.81956067901865E-02, 1.80455411334037E-02, 2.17820530671702E-02,
-0.400036026779312, -0.515038074910024, -0.758094294050284, 9.91972972972973E-04, 1, false, 4, 0);
            swFeatMgr.FeatureRevolve2(true, true, false, false, false, false, 0, 0, 6.28318530718, 0, false,
false, 0.01, 0.01, 0, 0, 0, true, true, true);
            swSelMgr.EnableContourSelection = false;
        }
    }
}
```

```
    }  
    public SldWorks swApp;  
  }  
}
```

Create Revolve Features Example (VBA)

This example shows how to create revolve features.

```

'-----
' Preconditions: Open a new part document.
'
' Postconditions: Two revolve features and one cut-revolve feature are created.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swFeatMgr As SldWorks.FeatureManager
Dim swSelMgr As SldWorks.SelectionMgr
Dim boolstatus As Boolean

Sub main()
    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swModelDocExt = swModel.Extension
    Set swSelMgr = swModel.SelectionManager

    ' Create an axis
    boolstatus = swModelDocExt.SelectByID2("Right Plane", "PLANE", 0, 0, 0, True, 0, Nothing, swSelectOptionDefault)
    boolstatus = swModelDocExt.SelectByID2("Top Plane", "PLANE", 0, 0, 0, True, 0, Nothing, swSelectOptionDefault)
    swModel.InsertAxis2 True

    ' Create a rectangle
    boolstatus = swModelDocExt.SelectByID2("Top Plane", "PLANE", -0.08954836342753, 4.336873289503E-04, 0.006720765739942, False, 0, Nothing, swSelectOptionDefault)
    swModel.InsertSketch2 True
    swModel.ClearSelection2 True
    swModel.SketchRectangle -0.05668466821757, -0.02198379306525, 0, -0.01330857427717, 0.03972855876814, 0, 1

    ' Create the first revolve feature
    swModel.InsertSketch2 True
    swModel.ShowNamedView2 "Trimetric", 8

    boolstatus = swModelDocExt.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, False, 0, Nothing, swSelectOptionDefault)
    boolstatus = swModelDocExt.SelectByID2("Axis1", "AXIS", 0, 0, 0, True, 16, Nothing, swSelectOptionDefault)
    Set swFeatMgr = swModel.FeatureManager

    swFeatMgr.FeatureRevolve2 True, True, False, False, False, False, 0, 0, 6.28318530718, 0, False, False, 0.01, 0.01, 0, 0, 0, True, True, True

    ' Create a cut-revolve feature using a face on the revolve feature
    swSelMgr.EnableContourSelection = 0
    boolstatus = swModelDocExt.SelectByID2("", "FACE", -0.03095803920934, 0.01509536510872, 0.02198379306526, False, 0, Nothing, swSelectOptionDefault)
    swModel.InsertSketch2 True
    swModel.ClearSelection2 True
    swModel.SketchRectangle -0.04194874421597, 0.01774859621099, 0, -0.01883036471929, -0.01265654504095, 0, 1
    swModel.InsertSketch2 True

    boolstatus = swModelDocExt.SelectByID2("Sketch2", "SKETCH", 0, 0, 0, False, 0, Nothing, swSelectOptionDefault)
    boolstatus = swModelDocExt.SelectByID2("Line4@Sketch2", "EXTSKETCHSEGMENT", -0.01883036471929, 0.003802500010693, 0, True, 4, Nothing, swSelectOptionDefault)
    swFeatMgr.FeatureRevolveCut 6.26573201466, False, 0, 0, 0, 1, 1

    ' Create the second revolve feature using a face on the first revolve feature
    swSelMgr.EnableContourSelection = 0
    boolstatus = swModelDocExt.SelectByID2("", "FACE", -0.02333512246603, 0.03472018719853, 0.0219837930652, False, 0, Nothing, swSelectOptionDefault)
    swModel.InsertSketch2 True
    swModel.ClearSelection2 True
    swModel.CreateCircle2 -0.02232361399104, 0.03354683337932, 0, -0.01445073476016, 0.02795861773112, 0
    swModel.InsertSketch2 True

    boolstatus = swModelDocExt.SelectByID2("Sketch3", "SKETCH", 0, 0, 0, False, 0, Nothing, swSelectOptionDefault)

    boolstatus = swModelDocExt.SelectByRay(-1.81956067901865E-02, 1.80455411334037E-02, 2.17820530671702E-02, -0.400036026779312, -0.515038074910024, -0.758094294050284, 9.91972972972973E-04, 1, False, 4, 0)

    swFeatMgr.FeatureRevolve2 True, True, False, False, False, False, 0, 0, 6.28318530718, 0, False, False, 0.01, 0.01, 0, 0, 0, True, True, True
    swSelMgr.EnableContourSelection = 0
End Sub

```

Create Revolve Features Example (VB.NET)

This example shows how to create revolve features.

```

'-----
' Preconditions: Open a new part document.
' Postconditions: Two revolve features and one cut-revolve feature are created.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System

Partial Class SolidWorksMacro

    Dim swModel As ModelDoc2
    Dim swModelDocExt As ModelDocExtension
    Dim swFeatMgr As FeatureManager
    Dim swSelMgr As SelectionMgr
    Dim boolstatus As Boolean

    Sub main()

        swModel = swApp.ActiveDoc
        swModelDocExt = swModel.Extension
        swSelMgr = swModel.SelectionManager

        ' Create an axis
        boolstatus = swModelDocExt.SelectByID2("Right Plane", "PLANE", 0, 0, 0, True, 0, Nothing,
swSelectOption_e.swSelectOptionDefault)
        boolstatus = swModelDocExt.SelectByID2("Top Plane", "PLANE", 0, 0, 0, True, 0, Nothing,
swSelectOption_e.swSelectOptionDefault)
        swModel.InsertAxis2(True)

        ' Create a rectangle
        boolstatus = swModelDocExt.SelectByID2("Top Plane", "PLANE", -0.08954836342753, 0.0004336873289503, 0.006720765739942,
False, 0, Nothing, swSelectOption_e.swSelectOptionDefault)
        swModel.InsertSketch2(True)
        swModel.ClearSelection2(True)
        swModel.SketchRectangle(-0.05668466821757, -0.02198379306525, 0, -0.01330857427717, 0.03972855876814, 0, 1)

        ' Create the first revolve feature
        swModel.InsertSketch2(True)
        swModel.ShowNamedView2("**Trimetric", 8)

        boolstatus = swModelDocExt.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, False, 0, Nothing,
swSelectOption_e.swSelectOptionDefault)
        boolstatus = swModelDocExt.SelectByID2("Axis1", "AXIS", 0, 0, 0, True, 16, Nothing, swSelectOption_e.swSelectOptionDefault)
        swFeatMgr = swModel.FeatureManager

        swFeatMgr.FeatureRevolve2(True, True, False, False, False, False, 0, 0, 6.28318530718, 0, False, False, 0.01, 0.01, 0, 0,
0, True, True, True)

        ' Create a cut-revolve feature using a face on the revolve feature
        swSelMgr.EnableContourSelection = 0
        boolstatus = swModelDocExt.SelectByID2("", "FACE", -0.03095803920934, 0.01509536510872, 0.02198379306526, False, 0,
Nothing, swSelectOption_e.swSelectOptionDefault)
        swModel.InsertSketch2(True)
        swModel.ClearSelection2(True)
        swModel.SketchRectangle(-0.04194874421597, 0.01774859621099, 0, -0.01883036471929, -0.01265654504095, 0, 1)
        swModel.InsertSketch2(True)

        boolstatus = swModelDocExt.SelectByID2("Sketch2", "SKETCH", 0, 0, 0, False, 0, Nothing,
swSelectOption_e.swSelectOptionDefault)
        boolstatus = swModelDocExt.SelectByID2("Line4@Sketch2", "EXTSKETCHSEGMENT", -0.01883036471929, 0.003802500010693, 0, True,
4, Nothing, swSelectOption_e.swSelectOptionDefault)
        swFeatMgr.FeatureRevolveCut(6.26573201466, False, 0, 0, 0, 1, 1)

        ' Create the second revolve feature using a face on the first revolve feature
        swSelMgr.EnableContourSelection = 0
        boolstatus = swModelDocExt.SelectByID2("", "FACE", -0.02333512246603, 0.03472018719853, 0.0219837930652, False, 0, Nothing,
swSelectOption_e.swSelectOptionDefault)
        swModel.InsertSketch2(True)
        swModel.ClearSelection2(True)
        swModel.CreateCircle2(-0.02232361399104, 0.03354683337932, 0, -0.01445073476016, 0.02795861773112, 0)
        swModel.InsertSketch2(True)

        boolstatus = swModelDocExt.SelectByID2("Sketch3", "SKETCH", 0, 0, 0, False, 0, Nothing,
swSelectOption_e.swSelectOptionDefault)
        boolstatus = swModelDocExt.SelectByRay(-1.81956067901865E-02, 1.80455411334037E-02, 2.17820530671702E-02,
-0.400036026779312, -0.515038074910024, -0.758094294050284, 9.91972972972973E-04, 1, False, 4, 0)
        swFeatMgr.FeatureRevolve2(True, True, False, False, False, False, 0, 0, 6.28318530718, 0, False, False, 0.01, 0.01, 0, 0,
0, True, True, True)
        swSelMgr.EnableContourSelection = 0

    End Sub

```

Public swApp As SldWorks

End Class

Create Ruled Surface Example (VBA)

This example shows how to create a ruled surface.

```
'-----  
' Preconditions:  
' 1. Open a part that has two edges created  
'    using b-curves.  
' 2. Select those two edges.  
' 3. Open the Immediate window.  
'  
' Postconditions:  
' 1. Creates a ruled surface.  
' 2. Examine the Immediate window.  
'-----  
Option Explicit  
  
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swModeler As SldWorks.Modeler  
    Dim swSelMgr As SldWorks.SelectionMgr  
    Dim swEdge(1) As SldWorks.Edge  
    Dim swCurve(1) As SldWorks.Curve  
    Dim nApex(2) As Double  
    Dim vApex As Variant  
    Dim swSurf As SldWorks.Surface  
  
    Set swApp = Application.SldWorks  
    Set swModeler = swApp.GetModeler  
    Set swModel = swApp.ActiveDoc  
    Set swSelMgr = swModel.SelectionManager  
    Set swEdge(0) = swSelMgr.GetSelectedObject5(1)  
    Set swCurve(0) = swEdge(0).GetCurve  
    Debug.Print "Is b-curve? " & swCurve(0).IsBcurve  
    Set swEdge(1) = swSelMgr.GetSelectedObject5(2)  
    Set swCurve(1) = swEdge(1).GetCurve  
    Debug.Print "Is b-curve? " & swCurve(1).IsBcurve  
    nApex(0) = 0#: nApex(1) = 0#: nApex(2) = 0#  
    vApex = nApex  
    Set swSurf = swModeler.CreateRuledSurface(swCurve(0), swCurve(1), (vApex))  
    Debug.Print "Type of surface? " & swSurf.Identity  
    Debug.Print "Is parametric? " & swSurf.IsParametric  
  
End Sub
```

Create Ruled Surface Body From Edge (VBA)

This example shows how to create a ruled surface body from an edge.

```
' -----
' Preconditions: Model document is open and an edge is
'               is selected on the model.
'
' Postconditions: Ruled surface body is created from the
'               selected edge.
' -----
```

Option Explicit

Public Enum swBodyType_e

```
    swAllBodies = 0
    swSolidBody = 1
    swSheetBody = 2
    swWireBody = 3
    swMinimumBody = 4
    swGeneralBody = 5
    swEmptyBody = 6
```

End Enum

```
Dim swApp As SldWorks.SldWorks
Dim swDoc As SldWorks.ModelDoc2
Dim swModel As SldWorks.Modeler
Dim swSelMgr As SldWorks.SelectionMgr
Dim swEdge(0) As SldWorks.Edge
Dim swEdgeList As Variant
Dim swRuledBody As SldWorks.Body2
```

Sub main()

```
Set swApp = Application.SldWorks
Set swDoc = swApp.ActiveDoc
Set swModel = swApp.GetModeler
Set swSelMgr = swDoc.SelectionManager
Set swEdge(0) = swSelMgr.GetSelectedObject6(1, -1)
swEdgeList = swEdge
Dim error As Long
error = swModel.CreateRuledSurfaceFromEdge(swDoc, swEdgeList, 1, 0.02, True, False, True, 0#, False, 1#, 0#, 0#, False,
swRuledBody)
Dim typeBody As Long
typeBody = swRuledBody.GetType()
Select Case typeBody
    Case swAllBodies
        Debug.Print "All solid and sheet bodies"
    Case swSolidBody
        Debug.Print "Solid body"
    Case swSheetBody
        Debug.Print "Sheet body"
    Case swWireBody
        Debug.Print "Wire body"
    Case swMinimumBody
        Debug.Print "Point body"
    Case swGeneralBody
        Debug.Print "General, nonmanifold body"
    Case swEmptyBody
        Debug.Print "NULL body"
```

End Select
End Sub

Create Save Bodies Feature and Create an Assembly Example (VBA)

This example shows how to create a save bodies feature, create part documents with the save bodies, and create an assembly with the parts.

```
' -----  
' Preconditions:  
' 1. Part document is open and contains a feature split of two.  
' 2. The Solid Bodies folder contains Split1[1] and Split2[2].  
' 3. C:\temp\SaveBodies exists.  
'  
' Postconditions:  
' 1. C:\temp\SaveBodies\1.sldprt, C:\temp\SaveBodies\2.sldprt,  
'    and C:\temp\assem.sldasm are created.  
' 2. In the original part, Save Bodies1 appears in the FeatureManager design tree.  
' -----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
```

```
Dim swModel As SldWorks.ModelDoc2
```

```
Dim swSelMgr As SldWorks.SelectionMgr
```

```
Dim swFeat As SldWorks.Feature
```

```
Dim swFeatMgr As SldWorks.FeatureManager
```

```
Dim swBodyFolder As SldWorks.BodyFolder
```

```
Dim v1 As Variant
```

```
Dim i As Long
```

```
Dim fileNames(1) As String
```

```
Dim fileNameVar As Variant
```

```
Sub GetVariantOfBody(swFeature As SldWorks.Feature, bodyList As Variant)
```

```
    Dim tt As Variant
```

```
    Set swBodyFolder = swFeature.GetSpecificFeature2
```

```
    Dim count As Integer
```

```
    count = swBodyFolder.GetBodyCount
```

```
    If (count < 1) Then
```

```
        MsgBox ("There are no bodies. Please create a body.")
```

```
    Else
```

```
        bodyList = swBodyFolder.GetBodies
```

End If

End Sub

Sub main()

Set swApp = Application.SldWorks

Set swModel = swApp.ActiveDoc

Set swSelMgr = swModel.SelectionManager

Set swFeat = swModel.FirstFeature

Set swFeatMgr = swModel.FeatureManager

Dim contLoop As Boolean

contLoop = True

While Not swFeat Is Nothing And contLoop = True

Dim Name As String

Name = swFeat.GetTypeName2

If (Name = "SolidBodyFolder") Then

GetVariantOfBody swFeat, v1

contLoop = False

End If

If (contLoop = True) Then

Set swFeat = swFeat.GetNextFeature

End If

Wend

fileNames(0) = "C:\temp\SaveBodies\1.sldprt"

fileNames(1) = "C:\temp\SaveBodies\2.sldprt"

fileNameVar = fileNames

swFeatMgr.**CreateSaveBodyFeature** v1, fileNameVar, "c:\temp\assem.sldasm", -1, -1

End Sub

Create Screw Mate Example (C#)

This example shows how to create a screw mechanical mate.

```
// -----
// Preconditions:
// 1. Open public_documents\samples\tutorial\api\MechanicalMates\Gearbox.sldasm.
// 2. Delete Screw2 from the Mates folder of the FeatureManager design tree.
//
// Postconditions: Inspect the Mates folder in the FeatureManager design tree.
//
// NOTE: Because the model is used elsewhere, do not save changes.
// -----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;

namespace CreateScrewMate_CSharp
{
    partial class SolidWorksMacro
    {
        private ModelDoc2 Part;
        private AssemblyDoc swAssembly;
        private bool boolstatus;

        public void Main()
        {
            Part = (ModelDoc2)swApp.ActiveDoc;
            swAssembly = (AssemblyDoc)Part;
            boolstatus = Part.Extension.SelectByRay(0.00613295661213442, 0.0689560082602441, 0.0745357419147012, -0.540837135649095,
            boolstatus = Part.Extension.SelectByRay(-0.000675539884355203, 0.0795322044607474, 0.0128285894949158, -0.540837135649095);

            // Create ScrewMateFeatureData
            ScrewMateFeatureData MateData = (ScrewMateFeatureData)swAssembly.CreateMateData(17);

            // Set the entities To mate
            object[] EntitiesToMate = new object[2];
            EntitiesToMate[0] = ((SelectionMgr)(Part.SelectionManager)).GetSelectedObject6(1, -1);
            EntitiesToMate[1] = ((SelectionMgr)(Part.SelectionManager)).GetSelectedObject6(2, -1);
            object EntitiesToMateVar = EntitiesToMate;
            MateData.EntitiesToMate = (EntitiesToMateVar);

            // Set the mate revolution type
            MateData.RevolutionType = 0;

            // Set the mate revolution value
            MateData.RevolutionVal = 1;

            // Set the mate orientation direction
            MateData.Reverse = true;

            // Create the mate
            Feature MateFeature = (Feature)swAssembly.CreateMate(MateData);
            Part.ClearSelection2(true);
            Part.EditRebuild3();
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Create Secondary Structure System Members Example (C#)

This example shows how to create two secondary structure system members: one between primary member pair end points and the other on a support plane between a primary member pair.

```
// =====
// Preconditions:
// 1. Ensure the specified part template exists.
// 2. Open an Immediate window.
// 3. Press F5.
//
// Postconditions:
// 1. Creates Sketch1 containing two sketch segments.
// 2. Configures the start/end extensions and the member profile.
// 3. Selects the two sketch segments.
// 4. Creates primary Structure System1 with two primary path segment members (Member1 and Member2).
// 5. Inspect the structure system in the graphics area.
// 6. Press F5 to add a secondary support plane member.
//    Creates secondary Structure System2 (Member3).
// 7. Press F5 to edit the support plane member.
// 8. Press F5 to add a secondary between points member.
//    Creates secondary Structure System3 (Member4).
// 9. Press F5 to edit the between points member.
//10. Press F5 to add a secondary structure system up to member.
//11. When the macro stops, multi-select a Member1 vertex
//     in the graphics area and Member2 in the FeatureManager Design tree.
//12. Press F5 to create the secondary Structure System4 (Member5).
//13. Inspect the Immediate window.
// =====
```

```
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Diagnostics;
using System.Runtime.InteropServices;
```

```
namespace Secondary
{
    public partial class SolidWorksMacro
    {
        private ModelDoc2 modDoc;
        private FeatureManager swFeatMgr;
        private SelectionMgr swSelMgr;
        private ModelDocExtension modDocExt;
        private StructureSystemMemberFeatureData structMemDef;
        private StructureSystemMemberProfile profDef;
        private PrimaryStructuralMemberFeatureData PrimDef;
        private PrimaryMemberPathSegmentFeatureData memPathSegDef;
        private SecondaryMemberBetweenPointsFeatureData memBetweenPointsSecDef;
        private SecondaryMemberBetweenPointsFeatureData secMemDatTochange;
        private SecondaryMemberSupportPlaneFeatureData memSupportPlaneSecDef;
        private object[] segments = new object[3];
        private bool stat;
        private StructureSystemMemberFeatureData[] memData = new StructureSystemMemberFeatureData[1];
        private object[] memDataArray;
        private Feature structSys;
        private StructureSystemFolder baseSecondarySystem;
        private StructureSystemFolder structSysDef;
        private StructureSystemFolder structSysSecDef;
        private Feature feat;
        private StructureSystemFolder structSysModDef;
        private object[] outProfiles;
```

```

private StructureSystemMemberFeatureData MemberData;
private object[] memberArray;
private Feature profileGrpFeat;
private ProfileGroupFolder profileGrp;
private StructureSystemMemberFeatureData memTochange;
private int i;
private int j;
private SecondaryStructuralMemberFeatureData secDef;
private SecondaryMemberUpToMembersFeatureData UpToMemDef;
private object point;
private object[] Members = new object[1];

public Dispatcher[] ObjectArrayToDispatcherArray(object[] Objects)
{
    int ArraySize;
    ArraySize = Objects.GetUpperBound(0);
    var d = new Dispatcher[ArraySize + 1];
    int ArrayIndex;
    var loopTo = ArraySize;
    for (ArrayIndex = 0; ArrayIndex <= loopTo; ArrayIndex++)
        d[ArrayIndex] = new Dispatcher(Objects[ArrayIndex]);
    return d;
}

public void Main()
{
    double swSheetWidth;
    swSheetWidth = 0d;
    double swSheetHeight;
    swSheetHeight = 0d;
    modDoc = (ModelDoc2)swApp.NewDocument(@"C:\ProgramData\SolidWorks\SOLIDWORKS 2023\templates\Part.PRTDOT", 0,
swSheetWidth, swSheetHeight);
    modDocExt = modDoc.Extension;
    modDoc.SketchManager.InsertSketch(true);
    stat = modDocExt.SelectByID2("Front Plane", "PLANE", -0.077188654347454d, 0.054268560279924d, 0.00386214196026222d,
false, 0, null, 0);
    modDoc.ClearSelection2(true);
    object skSegment;
    skSegment = modDoc.SketchManager.CreateLine(-0.168061d, 0.084736d, 0d, -0.168061d, -0.077684d, 0d);
    skSegment = modDoc.SketchManager.CreateLine(0.075216d, 0.107771d, 0d, 0.075216d, -0.006699d, 0d);
    modDoc.ClearSelection2(true);
    stat = modDocExt.SelectByID2("Top Plane", "PLANE", 0, 0, 0, true, 0, null, 0);
    RefPlane myRefPlane;
    myRefPlane = (RefPlane)modDoc.FeatureManager.InsertRefPlane(8, 0.03d, 0, 0, 0, 0);
    modDoc.ClearSelection2(true);
    stat = modDocExt.SelectByID2("Top Plane", "PLANE", 0, 0, 0, false, 0, null, 0);
    modDoc.ClearSelection2(true);
    modDoc.SketchManager.InsertSketch(true);

    stat = modDocExt.SelectByID2("Line1@Sketch1", "EXTSKETCHSEGMENT", -0.168061313304597d, 0.0544142573846353d, 0, false,
0, null, 0);
    stat = modDocExt.SelectByID2("Line2@Sketch1", "EXTSKETCHSEGMENT", 0.0752162521083513d, 0.0532390034454423d, 0, true, 0,
null, 0);

    // Create primary structure system path segment members
    structMemDef = (StructureSystemMemberFeatureData)modDocExt.CreateStructureSystemMemberData(0);
    Debug.Print("Type of structure system member as defined by swStructureSystemMemberType_e: " +
structMemDef.StructureSystemMemberType);
    structMemDef.StartEndExtendD1 = 1.0d;
    structMemDef.StartEndExtendD2 = 2.0d;
    profDef = (StructureSystemMemberProfile)structMemDef.MemberProfile;
    profDef.ProfileStandard = "ansi inch";
    profDef.ProfileType = "c channel";
    profDef.ProfileSize = "3 x 5";
    PrimDef = (PrimaryStructuralMemberFeatureData)structMemDef;

```

```

        Debug.Print("Structure system primary member creation type as defined by swStructureSystemMemberCreationType_e: " +
PrimDef.PrimaryMemberType);
        memPathSegDef = (PrimaryMemberPathSegmentFeatureData)PrimDef;
        memPathSegDef.MergeTangentMembers = true;
        swSelMgr = (SelectionMgr)modDoc.SelectionManager;
        var segments = new object[2];
        segments[0] = swSelMgr.GetSelectedObject6(1, 0);
        segments[1] = swSelMgr.GetSelectedObject6(2, 0);
        DispatchWrapper[] dArray;
        dArray = ObjectArrayToDispatchWrapperArray(segments);
        stat = memPathSegDef.SetPathSegments(dArray);
        Debug.Print("Path segments successfully set: " + stat);
        var PrimMemData = new StructureSystemMemberFeatureData[1];
        PrimMemData[0] = structMemDef;
        object[] PrimMemDatArray;
        PrimMemDatArray = PrimMemData;
        Feature structSysFeat;
        structSysFeat = modDocExt.CreateStructureSystem(PrimMemDatArray, null);
        modDoc.ViewZoomtofit();
        Debugger.Break();
        structSysDef = (StructureSystemFolder)structSysFeat.GetSpecificFeature2();
        Debug.Print("Number of profile group folders: " + structSysDef.GetProfileGroupFoldersCount());
        object[] outProfiles;
        outProfiles = (object[])structSysDef.GetProfileGroupFolders();
        var memberArray = default(object[]);
        if (outProfiles.Length > 0)
        {
            var loopTo = outProfiles.GetUpperBound(0);
            for (I = 0; I <= loopTo; I++)
            {
                profileGrpFeat = (Feature)outProfiles[I];
                profileGrp = (ProfileGroupFolder)profileGrpFeat.GetSpecificFeature2();
                if (profileGrp is object)
                {
                    Debug.Print("Number of members in profile group: " + profileGrp.GetStructureSystemMemberCount());
                    memberArray = (object[])profileGrp.GetStructureSystemMembers();
                    if (memberArray.Length > 0)
                    {
                        break;
                    }
                }
            }
        }

        // Create secondary structure system support plane member
        StructureSystemMemberFeatureData structSecMemDef;
        structSecMemDef = (StructureSystemMemberFeatureData)modDocExt.CreateStructureSystemMemberData(4); // Secondary Support
Plane
        Debug.Print("Type of structure system member as defined by swStructureSystemMemberType_e: " +
structMemDef.StructureSystemMemberType);
        structSecMemDef.StartEndExtendD1 = 0.5d;
        structSecMemDef.StartEndExtendD2 = 0.5d;
        StructureSystemMemberProfile secProfDef;
        secProfDef = (StructureSystemMemberProfile)structSecMemDef.MemberProfile;
        secProfDef.ProfileStandard = "ansi inch";
        secProfDef.ProfileType = "c channel";
        secProfDef.ProfileSize = "3 x 5";
        SecondaryStructuralMemberFeatureData SecDef;
        SecDef = (SecondaryStructuralMemberFeatureData)structSecMemDef;
        Debug.Print("Structure system secondary member creation type as defined by swStructureSystemMemberCreationType_e: " +
SecDef.SecondaryMemberType);
        memSupportPlaneSecDef = (SecondaryMemberSupportPlaneFeatureData)SecDef;
        DispatchWrapper[] eArray;
        eArray = ObjectArrayToDispatchWrapperArray(memberArray);
        stat = memSupportPlaneSecDef.SetMemberPairs(eArray);
        Debug.Print("Member pairs set successfully: " + stat);

```

```

stat = modDocExt.SelectByID2("Plane1", "PLANE", 0, 0, 0, true, 0, null, 0);
var Planes = new object[1];
Planes[0] = swSelMgr.GetSelectedObject6(1, 0);
DispatchWrapper[] fArray;
fArray = ObjectArrayToDispatchWrapperArray(Planes);
stat = memSupportPlaneSecDef.SetIntersectingPlanesAndFaces(fArray);
Debug.Print("Intersecting plane set successfully: " + stat);
var secMemData = new StructureSystemMemberFeatureData[1];
secMemData[0] = structSecMemDef;
object[] secMemDatArray;
secMemDatArray = secMemData;
Feature structSysFeat2;
structSysFeat2 = modDocExt.CreateStructureSystem(null, secMemDatArray);
modDoc.ViewZoomtofit();
Debugger.Break();

// Edit secondary support plane member by changing start/end extensions
baseSecondarySystem = (StructureSystemFolder)structSysFeat2.GetSpecificFeature2();
Debug.Print("Number of profile group folders: " + baseSecondarySystem.GetProfileGroupFoldersCount());
outProfiles = (object[])baseSecondarySystem.GetProfileGroupFolders();
var secMemberData = default(object);
object[] SecMemberArray;
Feature secProfileGrpFeat;
ProfileGroupFolder secProfileGrp;
if (outProfiles.Length > 0)
{
    var loopTo1 = outProfiles.GetUpperBound(0);
    for (I = 0; I <= loopTo1; I++)
    {
        secProfileGrpFeat = (Feature)outProfiles[I];
        secProfileGrp = (ProfileGroupFolder)secProfileGrpFeat.GetSpecificFeature2();
        if (secProfileGrp is object)
        {
            Debug.Print("Number of members in profile group: " + secProfileGrp.GetStructureSystemMemberCount());
            SecMemberArray = (object[])secProfileGrp.GetStructureSystemMembers();
            if (SecMemberArray.Length > 0)
            {
                var loopTo2 = SecMemberArray.GetUpperBound(0);
                for (j = 0; j <= loopTo2; j++)
                {
                    secMemberData = SecMemberArray[j];
                    break;
                }
            }
        }
    }
}

StructureSystemMemberFeatureData memToChange;
memToChange = (StructureSystemMemberFeatureData)secMemberData;
Feature SecMemFeat;
SecMemFeat = memToChange.GetFeature();
memToChange.StartEndExtendD1 = 0.2d;
memToChange.StartEndExtendD2 = 0.1d;
stat = SecMemFeat.ModifyDefinition(secMemberData, modDoc, null);
Debugger.Break();

// Create secondary between points member
structSecMemDef = (StructureSystemMemberFeatureData)modDocExt.CreateStructureSystemMemberData(5);
Debug.Print("Type of structure system member as defined by swStructureSystemMemberType_e: " +
structMemDef.StructureSystemMemberType);
secProfDef = (StructureSystemMemberProfile)structSecMemDef.MemberProfile;
secProfDef.ProfileStandard = "ansi inch";
secProfDef.ProfileType = "c channel";
secProfDef.ProfileSize = "3 x 5";

```

```

SecDef = (SecondaryStructuralMemberFeatureData)structSecMemDef;
Debug.Print("Structure system secondary member creation type as defined by swStructureSystemMemberCreationType_e: " +
SecDef.SecondaryMemberType);
memBetweenPointsSecDef = (SecondaryMemberBetweenPointsFeatureData)SecDef;
stat = memBetweenPointsSecDef.SetMemberPairs(eArray);
Debug.Print("Member pairs successfully set: " + stat);
memBetweenPointsSecDef.SecondaryMemberOffsetType = 0;
memBetweenPointsSecDef.DistanceMember1 = 0.5d;
memBetweenPointsSecDef.DistanceMember2 = 0.4d;
memBetweenPointsSecDef.RevDirectionDistanceMember1 = false;
memBetweenPointsSecDef.RevDirectionDistanceMember2 = false;
secMemData[0] = structSecMemDef;
secMemDatArray = secMemData;
structSysFeat2 = modDocExt.CreateStructureSystem(null, secMemDatArray);
Debugger.Break();

// Edit between points member by changing start/end extensions and offset type
structSysSecDef = (StructureSystemFolder)structSysFeat2.GetSpecificFeature2();
Debug.Print("Number of profile group folders: " + structSysSecDef.GetProfileGroupFoldersCount());
object[] outSecProfiles;
outSecProfiles = (object[])structSysSecDef.GetProfileGroupFolders();
if (outSecProfiles.Length > 0)
{
    var loopTo3 = outSecProfiles.GetUpperBound(0);
    for (I = 0; I <= loopTo3; I++)
    {
        secProfileGrpFeat = (Feature)outSecProfiles[I];
        secProfileGrp = (ProfileGroupFolder)secProfileGrpFeat.GetSpecificFeature2();
        if (secProfileGrp is object)
        {
            Debug.Print("Number of members in profile group: " + secProfileGrp.GetStructureSystemMemberCount());
            SecMemberArray = (object[])secProfileGrp.GetStructureSystemMembers();
            if (SecMemberArray.Length > 0)
            {
                var loopTo4 = SecMemberArray.GetUpperBound(0);
                for (j = 0; j <= loopTo4; j++)
                {
                    secMemberData = SecMemberArray[j];
                }
            }
        }
    }
}

memToChange = (StructureSystemMemberFeatureData)secMemberData;
SecMemFeat = memToChange.GetFeature();
memToChange.StartEndExtendD1 = 0.2d;
memToChange.StartEndExtendD2 = 0.1d;
secMemDatToChange = (SecondaryMemberBetweenPointsFeatureData)memToChange;
secMemDatToChange.SecondaryMemberOffsetType = 1;
secMemDatToChange.LengthRatioMember1 = 0.1d;
secMemDatToChange.LengthRatioMember2 = 0.2d;
stat = SecMemFeat.ModifyDefinition(secMemberData, modDoc, null);

Debugger.Break();

//Create secondary structure system up to member
structMemDef = modDocExt.CreateStructureSystemMemberData(6);
Debug.Print("Structure system member type as defined by swStructureSystemMemberType_e: " +
structMemDef.StructureSystemMemberType);

structMemDef.StartEndExtendD1 = 0.0;
Debug.Print("Start/end direction 1: " + structMemDef.StartEndExtendD1);
structMemDef.StartEndExtendD2 = 0.0;
Debug.Print("Start/end direction 2: " + structMemDef.StartEndExtendD2);

profDef = structMemDef.MemberProfile;

profDef.ProfileStandard = "ansi inch";
profDef.ProfileType = "pipe";
profDef.ProfileSize = "0.5 sch 40";

```

```

        secDef = (SecondaryStructuralMemberFeatureData)structMemDef;
        Debug.Print("Secondary member type as defined by swStructureSystemMemberCreationType_e: " +
secDef.SecondaryMemberType);

        UpToMemDef = (SecondaryMemberUpToMembersFeatureData)secDef;

        UpToMemDef.MemberPointParametersType = (int)
swSecondaryMemberUpToMembersMemberPointParameters_e.swSecondaryMemberUpToMembersMemberPointParameters_FromPoint;

        Debugger.Break(); //Multi-select a Member1 vertex in the graphics area and Member2 in the FeatureManager Design tree

        point = swSelMgr.GetSelectedObject6(1, 0);
        Members[0] = swSelMgr.GetSelectedObject6(2, 0);

        UpToMemDef.SetFromPoint(point);

        Dispatcher[] gArray;
        gArray = ObjectArrayToDispatcherArray(Members);
        UpToMemDef.SetFromPointMembers(gArray);

        UpToMemDef.SecondaryMemberOffsetType = (int)
swSecondaryMemberUpToMembersDistanceFromEndType_e.swSecondaryMemberUpToMembersDistanceFromEndType_Distance;
        UpToMemDef.DistanceMember = 0.1;
        UpToMemDef.RevDirectionDistanceMember = false;

        memData[0] = structMemDef;
        memDataArray = memData;

        Feature structSysFeat3;
        structSysFeat3 = modDocExt.CreateStructureSystem(null, memDataArray);
    }

    // The SldWorks swApp variable is pre-assigned for you.
    public SldWorks swApp;

}
}

```

Create Secondary Structure System Members Example (VB.NET)

This example shows how to create two secondary structure system members: one between primary member pair end points and the other on a support plane between a primary member pair.

```
'=====
'Preconditions:
'1. Ensure the specified part template exists.
'2. Open an Immediate window.
'3. Press F5.
'
'Postconditions:
' 1. Creates Sketch1 containing two sketch segments.
' 2. Configures the start/end extensions and the member profile.
' 3. Selects the two sketch segments.
' 4. Creates primary Structure System1 with two primary path segment members (Member1 and Member2).
' 5. Inspect the structure system in the graphics area.
' 6. Press F5 to add a secondary support plane member.
'    Creates secondary Structure System2 (Member3).
' 7. Press F5 to edit the support plane member.
' 8. Press F5 to add a secondary between points member.
'    Creates secondary Structure System3 (Member4).
' 9. Press F5 to edit the between points member.
'10. Press F5 to add a secondary structure system up to member.
'11. When the macro stops, multi-select a Member1 vertex
'    in the graphics area and Member2 in the FeatureManager Design tree.
'12. Press F5 to create the secondary Structure System4 (Member5).
'13. Inspect the Immediate window.
'=====
```

```
Imports SolidWorks.Interop.sldworks
```

```
Imports SolidWorks.Interop.swconst
```

```
Imports System.Runtime.InteropServices
```

```
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Dim modDoc As ModelDoc2
    Dim swFeatMgr As FeatureManager
    Dim swSelMgr As SelectionMgr
    Dim modDocExt As ModelDocExtension
    Dim structMemDef As StructureSystemMemberFeatureData
    Dim profDef As StructureSystemMemberProfile
    Dim PrimDef As PrimaryStructuralMemberFeatureData
    Dim memPathSegDef As PrimaryMemberPathSegmentFeatureData
    Dim memBetweenPointsSecDef As SecondaryMemberBetweenPointsFeatureData
    Dim secMemDatToChange As SecondaryMemberBetweenPointsFeatureData
    Dim memSupportPlaneSecDef As SecondaryMemberSupportPlaneFeatureData
    Dim segments(2) As Object
    Dim stat As Boolean
    Dim memData(0) As StructureSystemMemberFeatureData
    Dim memDataArray() As Object
    Dim structSys As Feature
    Dim baseSecondarySystem As StructureSystemFolder
    Dim structSysDef As StructureSystemFolder
    Dim structSysSecDef As StructureSystemFolder
    Dim feat As Feature
    Dim structSysModDef As StructureSystemFolder
    Dim outProfiles() As Object
    Dim MemberData As StructureSystemMemberFeatureData
    Dim memberArray() As Object
```

```

Dim profileGrpFeat As Feature
Dim profileGrp As ProfileGroupFolder
Dim memToChange As StructureSystemMemberFeatureData
Dim i As Integer
Dim j As Integer
Dim secDef As SecondaryStructuralMemberFeatureData
Dim upToMemDef As SecondaryMemberUpToMembersFeatureData
Dim point As Object
Dim Members(0) As Object

Function ObjectArrayToDispatchWrapperArray(ByVal Objects As Object()) As DispatchWrapper()
    Dim ArraySize As Integer
    ArraySize = Objects.GetUpperBound(0)
    Dim d(ArraySize) As DispatchWrapper
    Dim ArrayIndex As Integer
    For ArrayIndex = 0 To ArraySize
        d(ArrayIndex) = New DispatchWrapper(Objects(ArrayIndex))
    Next
    Return d
End Function

Sub main()

    Dim swSheetWidth As Double
    swSheetWidth = 0
    Dim swSheetHeight As Double
    swSheetHeight = 0
    modDoc = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2023\templates\Part.PRTDOT", 0, swSheetWidth,
swSheetHeight)

    modDocExt = modDoc.Extension

    modDoc.SketchManager.InsertSketch(True)
    stat = modDocExt.SelectByID2("Front Plane", "PLANE", -0.077188654347454, 0.054268560279924, 0.00386214196026222, False, 0,
Nothing, 0)
    modDoc.ClearSelection2(True)

    Dim skSegment As Object
    skSegment = modDoc.SketchManager.CreateLine(-0.168061, 0.084736, 0#, -0.168061, -0.077684, 0#)
    skSegment = modDoc.SketchManager.CreateLine(0.075216, 0.107771, 0#, 0.075216, -0.006699, 0#)
    modDoc.ClearSelection2(True)

    stat = modDocExt.SelectByID2("Top Plane", "PLANE", 0, 0, 0, True, 0, Nothing, 0)
    Dim myRefPlane As RefPlane
    myRefPlane = modDoc.FeatureManager.InsertRefPlane(8, 0.03, 0, 0, 0, 0)
    modDoc.ClearSelection2(True)
    stat = modDocExt.SelectByID2("Top Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
    modDoc.ClearSelection2(True)
    modDoc.SketchManager.InsertSketch(True)

    stat = modDocExt.SelectByID2("Line1@Sketch1", "EXTSKETCHSEGMENT", -0.168061313304597, 0.0544142573846353, 0, False, 0,
Nothing, 0)
    stat = modDocExt.SelectByID2("Line2@Sketch1", "EXTSKETCHSEGMENT", 0.0752162521083513, 0.0532390034454423, 0, True, 0,
Nothing, 0)

    'Create primary structure system path segment members
    structMemDef = modDocExt.CreateStructureSystemMemberData(0)
    Debug.Print("Type of structure system member as defined by swStructureSystemMemberType_e: " &
structMemDef.StructureSystemMemberType)

    structMemDef.StartEndExtendD1 = 1.0#
    structMemDef.StartEndExtendD2 = 2.0#

```

```

profDef = structMemDef.MemberProfile

profDef.ProfileStandard = "ansi inch"
profDef.ProfileType = "c channel"
profDef.ProfileSize = "3 x 5"

PrimDef = structMemDef
Debug.Print("Structure system primary member creation type as defined by swStructureSystemMemberCreationType_e: " &
PrimDef.PrimaryMemberType)

memPathSegDef = PrimDef
memPathSegDef.MergeTangentMembers = True

swSelMgr = modDoc.SelectionManager

Dim segments(1) As Object
segments(0) = swSelMgr.GetSelectedObject6(1, 0)
segments(1) = swSelMgr.GetSelectedObject6(2, 0)

Dim dArray() As DispatchWrapper
dArray = ObjectArrayToDispatchWrapperArray(segments)

stat = memPathSegDef.SetPathSegments(dArray)
Debug.Print("Path segments successfully set: " & stat)

Dim PrimMemData(0) As StructureSystemMemberFeatureData
PrimMemData(0) = structMemDef

Dim PrimMemDatArray() As Object
PrimMemDatArray = PrimMemData

Dim structSysFeat As Feature
structSysFeat = modDocExt.CreateStructureSystem(PrimMemDatArray, Nothing)

modDoc.ViewZoomtofit

Stop

structSysDef = structSysFeat.GetSpecificFeature2
Debug.Print("Number of profile group folders: " & structSysDef.GetProfileGroupFoldersCount)

Dim outProfiles() As Object
outProfiles = structSysDef.GetProfileGroupFolders()

Dim memberArray() As Object

If (outProfiles.Length > 0) Then
    For I = 0 To UBound(outProfiles)
        profileGrpFeat = outProfiles(I)
        profileGrp = profileGrpFeat.GetSpecificFeature2
        If Not profileGrp Is Nothing Then
            Debug.Print("Number of members in profile group: " & profileGrp.GetStructureSystemMemberCount)
            memberArray = profileGrp.GetStructureSystemMembers()
            If (memberArray.Length > 0) Then
                Exit For
            End If
        End If
    Next
End If

'Create secondary structure system support plane member

```

```

Dim structSecMemDef As StructureSystemMemberFeatureData
structSecMemDef = modDocExt.CreateStructureSystemMemberData(4) 'Secondary Support Plane

Debug.Print("Type of structure system member as defined by swStructureSystemMemberType_e: " &
structMemDef.StructureSystemMemberType)

structSecMemDef.StartEndExtendD1 = 0.5
structSecMemDef.StartEndExtendD2 = 0.5

Dim secProfDef As StructureSystemMemberProfile
secProfDef = structSecMemDef.MemberProfile

secProfDef.ProfileStandard = "ansi inch"
secProfDef.ProfileType = "c channel"
secProfDef.ProfileSize = "3 x 5"

Dim SecDef As SecondaryStructuralMemberFeatureData
SecDef = structSecMemDef
Debug.Print("Structure system secondary member creation type as defined by swStructureSystemMemberCreationType_e: " &
SecDef.SecondaryMemberType)

memSupportPlaneSecDef = SecDef

Dim eArray() As DispatchWrapper
eArray = ObjectArrayToDispatchWrapperArray(memberArray)

stat = memSupportPlaneSecDef.SetMemberPairs(eArray)
Debug.Print("Member pairs set successfully: " & stat)

stat = modDocExt.SelectByID2("Plane1", "PLANE", 0, 0, 0, True, 0, Nothing, 0)
Dim Planes(0) As Object
Planes(0) = swSelMgr.GetSelectedObject6(1, 0)

Dim fArray() As DispatchWrapper
fArray = ObjectArrayToDispatchWrapperArray(Planes)

stat = memSupportPlaneSecDef.SetIntersectingPlanesAndFaces(fArray)
Debug.Print("Intersecting plane set successfully: " & stat)

Dim secMemData(0) As StructureSystemMemberFeatureData
secMemData(0) = structSecMemDef

Dim secMemDatArray() As Object
secMemDatArray = secMemData

Dim structSysFeat2 As Feature
structSysFeat2 = modDocExt.CreateStructureSystem(Nothing, secMemDatArray)

modDoc.ViewZoomtofit

Stop

'Edit secondary support plane member by changing start/end extensions
baseSecondarySystem = structSysFeat2.GetSpecificFeature2
Debug.Print("Number of profile group folders: " & baseSecondarySystem.GetProfileGroupFoldersCount)

outProfiles = baseSecondarySystem.GetProfileGroupFolders()

Dim secMemberData As Object
Dim SecMemberArray() As Object
Dim secProfileGrpFeat As Feature

```

```

Dim secProfileGrp As ProfileGroupFolder
If (outProfiles.Length > 0) Then
    For I = 0 To UBound(outProfiles)
        secProfileGrpFeat = outProfiles(I)
        secProfileGrp = secProfileGrpFeat.GetSpecificFeature2
        If Not secProfileGrp Is Nothing Then
            Debug.Print("Number of members in profile group: " & secProfileGrp.GetStructureSystemMemberCount)
            SecMemberArray = secProfileGrp.GetStructureSystemMembers()
            If (SecMemberArray.Length > 0) Then
                For j = 0 To UBound(SecMemberArray)
                    secMemberData = SecMemberArray(j)
                    Exit For
                Next
            End If
        End If
    Next
End If

Dim memToChange As StructureSystemMemberFeatureData
memToChange = secMemberData
Dim SecMemFeat As Feature
SecMemFeat = memToChange.GetFeature()
memToChange.StartEndExtendD1 = 0.2
memToChange.StartEndExtendD2 = 0.1

stat = SecMemFeat.ModifyDefinition(secMemberData, modDoc, Nothing)

Stop

'Create secondary between points member
structSecMemDef = modDocExt.CreateStructureSystemMemberData(5)
Debug.Print("Type of structure system member as defined by swStructureSystemMemberType_e: " &
structMemDef.StructureSystemMemberType)

secProfDef = structSecMemDef.MemberProfile

secProfDef.ProfileStandard = "ansi inch"
secProfDef.ProfileType = "c channel"
secProfDef.ProfileSize = "3 x 5"

SecDef = structSecMemDef
Debug.Print("Structure system secondary member creation type as defined by swStructureSystemMemberCreationType_e: " &
SecDef.SecondaryMemberType)

memBetweenPointsSecDef = SecDef

stat = memBetweenPointsSecDef.SetMemberPairs(eArray)
Debug.Print("Member pairs successfully set: " & stat)

memBetweenPointsSecDef.SecondaryMemberOffsetType = 0
memBetweenPointsSecDef.DistanceMember1 = 0.5
memBetweenPointsSecDef.DistanceMember2 = 0.4
memBetweenPointsSecDef.RevDirectionDistanceMember1 = False
memBetweenPointsSecDef.RevDirectionDistanceMember2 = False

secMemData(0) = structSecMemDef

secMemDatArray = secMemData

structSysFeat2 = modDocExt.CreateStructureSystem(Nothing, secMemDatArray)

Stop

```

```

'Edit between points member by changing start/end extensions and offset type
structSysSecDef = structSysFeat2.GetSpecificFeature2
Debug.Print("Number of profile group folders: " & structSysSecDef.GetProfileGroupFoldersCount)

Dim outSecProfiles() As Object
outSecProfiles = structSysSecDef.GetProfileGroupFolders()

If (outSecProfiles.Length > 0) Then
    For I = 0 To UBound(outSecProfiles)
        secProfileGrpFeat = outSecProfiles(I)
        secProfileGrp = secProfileGrpFeat.GetSpecificFeature2
        If Not secProfileGrp Is Nothing Then
            Debug.Print("Number of members in profile group: " & secProfileGrp.GetStructureSystemMemberCount)
            SecMemberArray = secProfileGrp.GetStructureSystemMembers()
            If (SecMemberArray.Length > 0) Then
                For j = 0 To UBound(SecMemberArray)
                    secMemberData = SecMemberArray(j)
                Next
            End If
        End If
    Next
End If

memToChange = secMemberData

SecMemFeat = memToChange.GetFeature()
memToChange.StartEndExtendD1 = 0.2
memToChange.StartEndExtendD2 = 0.1

secMemDatToChange = memToChange
secMemDatToChange.SecondaryMemberOffsetType = 1
secMemDatToChange.LengthRatioMember1 = 0.1
secMemDatToChange.LengthRatioMember2 = 0.2
stat = SecMemFeat.ModifyDefinition(secMemberData, modDoc, Nothing)
'Create secondary structure system up to member

structMemDef = modDocExt.CreateStructureSystemMemberData(6)

Debug.Print("Structure system member type as defined by swStructureSystemMemberType_e: " &
structMemDef.StructureSystemMemberType)

structMemDef.StartEndExtendD1 = 0.0
Debug.Print("Start/end direction 1: " & structMemDef.StartEndExtendD1)
structMemDef.StartEndExtendD2 = 0.0
Debug.Print("Start/end direction 2: " & structMemDef.StartEndExtendD2)

profDef = structMemDef.MemberProfile

profDef.ProfileStandard = "ansi inch"
profDef.ProfileType = "pipe"
profDef.ProfileSize = "0.5 sch 40"

SecDef = structMemDef
Debug.Print("Secondary member type as defined by swStructureSystemMemberCreationType_e: " & SecDef.SecondaryMemberType)

UpToMemDef = SecDef

UpToMemDef.MemberPointParametersType =
swSecondaryMemberUpToMembersMemberPointParameters_e.swSecondaryMemberUpToMembersMemberPointParameters_FromPoint

Stop 'Multi-Select a Member1 vertex In the graphics area And Member2 In the FeatureManager Design tree

point = swSelMgr.GetSelectedObject6(1, 0)

```

```

Members(0) = swSelMgr.GetSelectedObject6(2, 0)

UpToMemDef.SetFromPoint(point)

Dim gArray() As DispatchWrapper
gArray = ObjectArrayToDispatchWrapperArray(Members)
UpToMemDef.SetFromPointMembers(gArray)

UpToMemDef.SecondaryMemberOffsetType =
swSecondaryMemberUpToMembersDistanceFromEndType_e.swSecondaryMemberUpToMembersDistanceFromEndType_Distance
UpToMemDef.DistanceMember = 0.1
UpToMemDef.RevDirectionDistanceMember = False

memData(0) = structMemDef
memDataArray = memData

Dim structSysFeat3 As Feature
structSysFeat3 = modDocExt.CreateStructureSystem(Nothing, memDataArray)

End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
End Class

```

Create Section View and Get Some Data Example (C#)

This example creates a section view and sets and gets some of the section view's data.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\driveworksxpess\mobile gantry.slddrw
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Creates a section view of Drawing View4.
// 2. Sets and gets some section view settings.
// 3. Examine the drawing and the Immediate window.
//
// NOTE: Because this drawing is used elsewhere, do not save changes.
//-----

using System;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;

namespace CreateSectionView_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        DrawingDoc swDrawing;
        SketchManager swSketchMgr;
        SketchSegment swSketchSegment;
        object excludedComponents;
        View swView;
        DrSection swSectionView;

        bool boolstatus;

        public void Main()
        {
            swModel = (ModelDoc2)swApp.ActiveDoc;
            swDrawing = (DrawingDoc)swModel;

            // Activate the view for which you want to create a section view
            boolstatus = swDrawing.ActivateView("Drawing View4");
            swModel.ClearSelection2(true);

            // Create section-view line
            swSketchMgr = swModel.SketchManager;
            swSketchSegment = swSketchMgr.CreateLine(-1.383705, 2.078706, 0.0, 2.747162, 0.0441, 0.0);

            // Create the section view at the specified coordinates
            // and up to the specified distance from the section-view line
            excludedComponents = null;
            swView = swDrawing.CreateSectionViewAt5(0.1604082711061, 0.2048687170364, 0, "D", 32, (excludedComponents), 0.00835);
            Debug.Print("View data: ");
            Debug.Print(" Emphasize outlines of section views? " + swView.EmphasizeOutline);
            swSectionView = (DrSection)swView.GetSection();

            // Set some section-view settings
            swSectionView.SetAutoHatch(true);
            swSectionView.SetLabel2("ABCD");
            swSectionView.SetDisplayOnlySurfaceCut(false);
            swSectionView.SetPartialSection(false);
            swSectionView.SetReversedCutDirection(false);
            swSectionView.SetScaleWithModelChanges(true);
            swSectionView.CutSurfaceBodies = true;
            swSectionView.DisplaySurfaceBodies = true;
            swSectionView.ExcludeSliceSectionBodies = false;

            // Get some section-view settings
            Debug.Print("Section view data: ");
            Debug.Print(" Label: " + swSectionView.GetLabel());
            Debug.Print(" Name of section line: " + swSectionView.GetName());
            Debug.Print(" Depth: " + swSectionView.SectionDepth * 1000.0 + " mm");
            Debug.Print(" Cut direction reversed from default direction? " + swSectionView.GetReversedCutDirection());
            Debug.Print(" Partial section cut? " + swSectionView.GetPartialSection());
            Debug.Print(" Display only the surface cut by the section line? " + swSectionView.GetDisplayOnlySurfaceCut());
            Debug.Print(" Display surface bodies? " + swSectionView.DisplaySurfaceBodies);
            Debug.Print(" Exclude slice section bodies? " + swSectionView.ExcludeSliceSectionBodies);

            swSectionView.SetDisplayOnlySpeedPakBodies(true);

            Debug.Print(" Display only SpeedPak bodies? " + swSectionView.GetDisplayOnlySpeedPakBodies());
            Debug.Print(" Scale with model changes? " + swSectionView.GetScaleWithModelChanges());
            Debug.Print(" Auto-hatch enabled? " + swSectionView.GetAutoHatch());
            Debug.Print(" Hide cut surface bodies? " + swSectionView.CutSurfaceBodies);
        }
    }
}
```

```
swModel.EditRebuild3();  
    }  
    public SldWorks swApp;  
}  
}
```

Create Section View and Get Some Data Example (VBA)

This example creates a section view and sets and gets some of the section view's data.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\driveworks\express\mobile gantry.slddrw
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Creates a section view of Drawing View4.
' 2. Sets and gets some section view settings.
' 3. Examine the drawing and the Immediate window.
'
' NOTE: Because this drawing is used elsewhere, do not save changes.
'-----

```

Option Explicit

```

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swDrawing As SldWorks.DrawingDoc
Dim swSketchMgr As SldWorks.SketchManager
Dim swSketchSegment As SldWorks.SketchSegment
Dim excludedComponents As Variant
Dim swView As SldWorks.View
Dim swSectionView As SldWorks.DrSection
Dim boolstatus As Boolean

Sub main()
    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swDrawing = swModel

    ' Activate the view for which you want to create a section view
    boolstatus = swDrawing.ActivateView("Drawing View4")
    swModel.ClearSelection2 True

    ' Create section-view line
    Set swSketchMgr = swModel.SketchManager
    Set swSketchSegment = swSketchMgr.CreateLine(-1.383705, 2.078706, 0#, 2.747162, 0.0441, 0#)

    ' Create the section view at the specified coordinates
    ' and up to the specified distance from the section-view line
    Set excludedComponents = Nothing
    Set swView = swDrawing.CreateSectionViewAt5(0.1604082711061, 0.2048687170364, 0, "D", 32, (excludedComponents), 0.00835)
    Debug.Print "View data:"
    Debug.Print " Emphasize outlines of section views? " & swView.EmphasizeOutline
    Set swSectionView = swView.GetSection

    ' Set some section-view settings
    swSectionView.SetAutoHatch True
    swSectionView.SetLabel2 "ABCD"
    swSectionView.SetDisplayOnlySurfaceCut False
    swSectionView.SetPartialSection False
    swSectionView.SetReversedCutDirection False
    swSectionView.SetScaleWithModelChanges True
    swSectionView.CutSurfaceBodies = True
    swSectionView.DisplaySurfaceBodies = True
    swSectionView.ExcludeSliceSectionBodies = False

    ' Get some section-view settings
    Debug.Print "Section view data: "
    Debug.Print " Label: " & swSectionView.GetLabel
    Debug.Print " Name of section line: " & swSectionView.GetName
    Debug.Print " Depth: " & swSectionView.SectionDepth * 1000# & " mm"
    Debug.Print " Cut direction reversed from default direction? " & swSectionView.GetReversedCutDirection
    Debug.Print " Partial section cut? " & swSectionView.GetPartialSection
    Debug.Print " Display only the surface cut by the section line? " & swSectionView.GetDisplayOnlySurfaceCut
    Debug.Print " Display surface bodies? " & swSectionView.DisplaySurfaceBodies
    Debug.Print " Exclude slice section bodies? " & swSectionView.ExcludeSliceSectionBodies

    swSectionView.SetDisplayOnlySpeedPakBodies True

    Debug.Print " Display only SpeedPak bodies? " & swSectionView.GetDisplayOnlySpeedPakBodies
    Debug.Print " Get scale with model changes? " & swSectionView.GetScaleWithModelChanges
    Debug.Print " Auto-hatch enabled? " & swSectionView.GetAutoHatch
    Debug.Print " Hide cut surface bodies? " & swSectionView.CutSurfaceBodies

    swModel.EditRebuild3
End Sub

```

Create Section View and Get Some Data Example (VB.NET)

This example creates a section view and sets and gets some of the section view's data.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\driveworksxpess\mobile gantry.slddrw
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Creates a section view of Drawing View4.
' 2. Sets and gets some section view settings.
' 3. Examine the drawing and the Immediate window.
'
' NOTE: Because this drawing is used elsewhere, do not save changes.
'-----

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Dim swModel As ModelDoc2
    Dim swDrawing As DrawingDoc
    Dim swSketchMgr As SketchManager
    Dim swSketchSegment As SketchSegment
    Dim excludedComponents As Object
    Dim swView As View
    Dim swSectionView As DrSection
    Dim boolstatus As Boolean

    Sub main()

        swModel = swApp.ActiveDoc
        swDrawing = swModel

        ' Activate the view for which you want to create a section view
        boolstatus = swDrawing.ActivateView("Drawing View4")
        swModel.ClearSelection2(True)

        ' Create section-view line
        swSketchMgr = swModel.SketchManager
        swSketchSegment = swSketchMgr.CreateLine(-1.383705, 2.078706, 0.0#, 2.747162, 0.0441, 0.0#)

        ' Create the section view at the specified coordinates
        ' and up to the specified distance from the section-view line
        excludedComponents = Nothing
        swView = swDrawing.CreateSectionViewAt5(0.1604082711061, 0.2048687170364, 0, "D", 32, (excludedComponents), 0.00835)
        Debug.Print("View data:")
        Debug.Print(" Emphasize outlines of section views? " & swView.EmphasizeOutline)
        swSectionView = swView.GetSection

        ' Set some section-view settings
        swSectionView.SetAutoHatch(True)
        swSectionView.SetLabel2("ABCD")
        swSectionView.SetDisplayOnlySurfaceCut(False)
        swSectionView.SetPartialSection(False)
        swSectionView.SetReversedCutDirection(False)
        swSectionView.SetScaleWithModelChanges(True)
        swSectionView.CutSurfaceBodies = True
        swSectionView.DisplaySurfaceBodies = True
        swSectionView.ExcludeSliceSectionBodies = False

        ' Get some section-view settings
        Debug.Print("Section view data: ")
        Debug.Print(" Label: " & swSectionView.GetLabel)
        Debug.Print(" Name of section line: " & swSectionView.GetName)
        Debug.Print(" Depth: " & swSectionView.SectionDepth * 1000.0# & " mm")
        Debug.Print(" Cut direction reversed from default direction? " & swSectionView.GetReversedCutDirection)
        Debug.Print(" Partial section cut? " & swSectionView.GetPartialSection)
        Debug.Print(" Display only the surface cut by the section line? " & swSectionView.GetDisplayOnlySurfaceCut)
        Debug.Print(" Display surface bodies? " & swSectionView.DisplaySurfaceBodies)
        Debug.Print(" Exclude slice section bodies? " & swSectionView.ExcludeSliceSectionBodies)

        swSectionView.SetDisplayOnlySpeedPakBodies(True)

        Debug.Print(" Display only SpeedPak bodies? " & swSectionView.GetDisplayOnlySpeedPakBodies)
        Debug.Print(" Scale with model changes? " & swSectionView.GetScaleWithModelChanges)
        Debug.Print(" Auto-hatch enabled? " & swSectionView.GetAutoHatch)
        Debug.Print(" Hide cut surface bodies? " & swSectionView.CutSurfaceBodies)

        swModel.EditRebuild3()

    End Sub

End Class

```

```
Public swApp As SldWorks
```

```
End Class
```

Create Section View at Specified Location Example (VBA)

This example shows how to create a section view at a specified location.

```
'-----
' Preconditions: Open public_documents\samples\tutorial\advdrawings\foodprocessor.slddrw.
'
' Postconditions:
' 1. Creates a section view created at the specified location.
' 2. Examine the graphics area.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swDrawing As SldWorks.DrawingDoc
Dim vChildComponents As Variant
Dim swFirstDrawingView As SldWorks.View
Dim swDrawingView As SldWorks.View
Dim swDrRootComponent As SldWorks.DrawingComponent

Sub main()

    Set swApp = Application.SldWorks
    Set swDrawing = swApp.ActiveDoc
    If swDrawing Is Nothing Then
        MsgBox "There is no active document"
        Exit Sub
    End If

    ' Always returns the drawing sheet first
    Set swFirstDrawingView = swDrawing.GetFirstView

    ' Get first drawing view
    Set swDrawingView = swFirstDrawingView.GetNextView

    ' Get root drawing component
    Set swDrRootComponent = swDrawingView.RootDrawingComponent
    Dim iChild As Long
    If swDrRootComponent.GetChildrenCount > 0 Then
        vChildComponents = swDrRootComponent.GetChildren
        For iChild = 0 To UBound(vChildComponents)
            Next iChild
    End If

    ' Select the drawing view and create line for the section view
    Dim boolstatus As Boolean
    boolstatus = swDrawing.Extension.SelectByID2("Drawing View1", "DRAWINGVIEW", 0, 0, 0, False, 0, Nothing, swSelectOptionDefault)
    swDrawing.ClearSelection2 True
    swDrawing.CreateLine2 0.091354, 0.284441, 0#, 0.253455, 0.140062, 0#

    ' Set arguments for call to create section view
    Dim x As Double, y As Double, z As Double
    Dim notAligned As Boolean, isOffsetSection As Boolean, chgdirection As Boolean, scwithmodel As Boolean, partial As Boolean, dispsurfcut As Boolean
    Dim label As String
    Dim sectionView As SldWorks.View
    x = 0.65
    y = 0.08908737704918
    z = 0
    notAligned = False
    isOffsetSection = False
    chgdirection = False
    scwithmodel = False
    partial = False
    dispsurfcut = False
    label = "A"

    ' Create section view
    Set sectionView = swDrawing.CreateSectionViewAt3(x, y, z, notAligned, isOffsetSection, label, chgdirection, scwithmodel, partial, dispsurfcut,
(vChildComponents))

End Sub
```

Create Section View in Model Example (C#)

This example shows how to create a section view in a model.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\multibody\multi_bridge.sldprt.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Displays a section view with three sections that
//    are capped in color.
// 2. Examine the graphics area.
//
// NOTE: Because the part is used elsewhere, do not save changes.
//-----

using SolidWorks.Interop.sldworks;

using SolidWorks.Interop.swconst;

using System.Runtime.InteropServices;

using System;

using System.Diagnostics;

using Microsoft.VisualBasic;

namespace GraphicsOnlySectionView_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        ModelDocExtension swModelDocExt;
        ModelViewManager swModelViewMgr;
        SelectionMgr swSelMgr;
        SectionViewData svData;
        Feature plane1;
        Feature plane2;
        Feature plane3;

        bool boolstatus;

        public void Main()
        {
            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelDocExt = swModel.Extension;
            swModelViewMgr = swModel.ModelViewManager;

            swSelMgr = (SelectionMgr)swModel.SelectionManager;

            boolstatus = swModelDocExt.SelectByID("Front", "PLANE",
            -0.04751707843116, 0.04466659468449, 0.120999999999999, false, 1, null, 0);

            boolstatus = swModelDocExt.SelectByID("Top", "PLANE",
            -0.06781533387408, 0.00100317525596, 0.1263684575364, true, 2, null, 1);

            boolstatus = swModelDocExt.SelectByID("Right", "PLANE",
            -0.000808330303073, 0.07304457560201, -0.003890984556108, true, 4, null, 1);

            svData = swModelViewMgr.CreateSectionViewData();
            FillPlaneData(svData, swSelMgr);

            boolstatus = swModelViewMgr.CreateSectionView(svData);

            Debug.Print("
Section bodies are valid: " + boolstatus);
        }
    }
}
```

```

public void FillPlaneData(SectionViewData data, SelectionMgr selMgr)
{
    plane1 = (Feature)selMgr.GetSelectedObject6(1,
0);
    plane2 = (Feature)selMgr.GetSelectedObject6(2,
0);
    plane3 = (Feature)selMgr.GetSelectedObject6(4,
0);

    data.FirstPlane = plane1;
    data.FirstReverseDirection = false;
    data.FirstOffset = -0.01;
    data.FirstRotationX = 0.296705972839036;
    data.FirstRotationY = 0.174532925199433;
    data.FirstColor = Information.RGB(255, 0, 0);

    data.SecondPlane = plane2;
    data.SecondReverseDirection = false;
    data.SecondOffset = 0.01;
    data.SecondRotationX = 0.296705972839036;
    data.SecondRotationY = 0.174532925199433;
    data.SecondColor = Information.RGB(0, 255, 0);

    data.ThirdPlane = plane3;
    data.ThirdReverseDirection = true;
    data.ThirdOffset = -0.01;
    data.ThirdRotationX = 0.296705972839036;
    data.ThirdRotationY = 0.174532925199433;
    data.ThirdColor = Information.RGB(0, 0, 255);

    data.Redraw = true;
    data.ShowSectionCap = true;
    data.KeepCapColor = true;
    data.GraphicsOnlySection = true;

}

public SldWorks swApp;

}

}

```

Create Section View in Model Example (VBA)

This example shows how to create a section view in a model.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\multibody\multi_bridge.sldprt.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Displays a section view with three sections that
'    are capped in color.
' 2. Examine the graphics area.
'
' NOTE: Because the part is used elsewhere, do not save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swModelViewMgr As SldWorks.ModelViewManager
Dim swSelMgr As SldWorks.SelectionMgr
Dim svData As SldWorks.SectionViewData
Dim plane1 As SldWorks.Feature, plane2 As SldWorks.Feature, plane3 As SldWorks.Feature
Dim boolstatus As Boolean

Sub main()
    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swModelDocExt = swModel.Extension
    Set swModelViewMgr = swModel.ModelViewManager
    Set swSelMgr = swModel.SelectionManager

    boolstatus = swModelDocExt.SelectByID2("Front", "PLANE", -0.04751707843116, 0.04466659468449, 0.120999999999999, False, 1,
Nothing, 0)
    boolstatus = swModelDocExt.SelectByID2("Top", "PLANE", -0.06781533387408, 0.00100317525596, 0.1263684575364, True, 2, Nothing,
1)
    boolstatus = swModelDocExt.SelectByID2("Right", "PLANE", -0.000808330303073, 0.07304457560201, -0.003890984556108, True, 4,
Nothing, 1)

    Set svData = swModelViewMgr.CreateSectionViewData()
    FillPlaneData svData, swSelMgr

    boolstatus = swModelViewMgr.CreateSectionView(svData)
    Debug.Print " Section bodies are valid: " & boolstatus
End Sub

Sub FillPlaneData(data As SectionViewData, selMgr As SelectionMgr)

    Set plane1 = selMgr.GetSelectedObject6(1, 0)
    Set plane2 = selMgr.GetSelectedObject6(2, 0)
    Set plane3 = selMgr.GetSelectedObject6(4, 0)

    data.FirstPlane = plane1
    data.FirstReverseDirection = False
    data.FirstOffset = -0.01
    data.FirstRotationX = 0.296705972839036
    data.FirstRotationY = 0.174532925199433
    data.FirstColor = RGB(255, 0, 0)

    data.SecondPlane = plane2
    data.SecondReverseDirection = False
    data.SecondOffset = 0.01
    data.SecondRotationX = 0.296705972839036
    data.SecondRotationY = 0.174532925199433
    data.SecondColor = RGB(0, 255, 0)

    data.ThirdPlane = plane3
    data.ThirdReverseDirection = True
    data.ThirdOffset = -0.01
    data.ThirdRotationX = 0.296705972839036
    data.ThirdRotationY = 0.174532925199433
    data.ThirdColor = RGB(0, 0, 255)

    data.Redraw = True
    data.ShowSectionCap = True
    data.KeepCapColor = True
    data.GraphicsOnlySection = True
End Sub

```

Create Section View in Model Example (VB.NET)

This example shows how to create a section view in a model.

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\multibody\multi_bridge.sldprt.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Displays a section view with three sections that
'   are capped in color.
' 2. Examine the graphics area.
'
' NOTE: Because the part is used elsewhere, do not save changes.
'-----
```

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Dim swModel As ModelDoc2
    Dim swModelDocExt As ModelDocExtension
    Dim swModelViewMgr As ModelViewManager
    Dim swSelMgr As SelectionMgr
    Dim svData As SectionViewData
    Dim plane1 As Feature, plane2 As Feature, plane3 As Feature
    Dim boolstatus As Boolean

    Sub main()

        swModel = swApp.ActiveDoc
        swModelDocExt = swModel.Extension
        swModelViewMgr = swModel.ModelViewManager

        swSelMgr = swModel.SelectionManager

        boolstatus = swModelDocExt.SelectByID("Front", "PLANE",
        -0.04751707843116, 0.04466659468449, 0.120999999999999, False, 1, Nothing,
        0)

        boolstatus = swModelDocExt.SelectByID("Top", "PLANE",
        -0.06781533387408, 0.00100317525596, 0.1263684575364, True, 2, Nothing,
        1)

        boolstatus = swModelDocExt.SelectByID("Right", "PLANE",
        -0.000808330303073, 0.07304457560201, -0.003890984556108, True, 4, Nothing,
        1)

        svData = swModelViewMgr.CreateSectionViewData()
        FillPlaneData(svData, swSelMgr)

        boolstatus = swModelViewMgr.CreateSectionView(svData)

        Debug.Print(" Section
bodies are valid: " & boolstatus)

    End Sub

    Sub FillPlaneData(ByVal data As SectionViewData, ByVal selMgr As SelectionMgr)

        plane1 = selMgr.GetSelectedObject6(1, 0)
        plane2 = selMgr.GetSelectedObject6(2, 0)
        plane3 = selMgr.GetSelectedObject6(4, 0)
```

```

        data.FirstPlane = plane1
        data.FirstReverseDirection = False
        data.FirstOffset =
-0.01
        data.FirstRotationX = 0.296705972839036
        data.FirstRotationY = 0.174532925199433
        data.FirstColor = RGB(255, 0, 0)

        data.SecondPlane = plane2
        data.SecondReverseDirection = False
        data.SecondOffset =
0.01
        data.SecondRotationX = 0.296705972839036
        data.SecondRotationY = 0.174532925199433
        data.SecondColor = RGB(0, 255, 0)

        data.ThirdPlane = plane3
        data.ThirdReverseDirection = True
        data.ThirdOffset =
-0.01
        data.ThirdRotationX = 0.296705972839036
        data.ThirdRotationY = 0.174532925199433
        data.ThirdColor = RGB(0, 0, 255)

        data.Redraw = True
        data.ShowSectionCap = True
        data.KeepCapColor = True
        data.GraphicsOnlySection = True

End Sub

Public swApp As SldWorks

End Class

```

Create Sheet Body From Faces and Feature from Sheet Body Example (VB)

This example shows how to create a sheet body from the faces of a solid body and how to create a feature from the sheet body.

```
'-----  
'  
' Preconditions: Part document is open that contains  
'     a solid body named Extrude1.  
'  
' Postconditions: Surface-Imported feature is created from  
'     the sheet body.  
'  
'-----
```

Option Explicit

```
Dim swApp           As SldWorks.SldWorks  
Dim swModel         As SldWorks.ModelDoc2  
Dim swModeler       As SldWorks.Modeler  
Dim swBody          As SldWorks.Body2  
Dim swFace          As SldWorks.Face2  
Dim swWorkPieceBody As SldWorks.Body2  
Dim swWorkPieceFacesBody As SldWorks.Body2  
Dim swWorkPieceBodyCopy As SldWorks.Body2  
Dim aFaces()        As SldWorks.Face2  
Dim swThickenedBody As SldWorks.Body2  
Dim swFeatureManager As SldWorks.FeatureManager  
Dim aBodies(0)      As SldWorks.Body2  
Dim vBodies          As Variant  
Dim swPart           As PartDoc  
Dim swFeature        As SldWorks.Feature  
Dim bValue           As Boolean  
Dim lErrors          As Long  
Dim lWarnings        As Long  
Dim vFaces           As Variant  
Dim lIdx             As Long  
Dim retVal           As Long
```

Sub main()

```
' Connect to SOLIDWORKS Design  
Set swApp = Application.SldWorks  
Set swModel = swApp.ActiveDoc  
Set swFeatureManager = swModel.FeatureManager  
  
' Get Modeler object  
Set swModeler = swApp.GetModeler  
  
' Clear selection  
swModel.ClearSelection2 True  
  
' Select the body  
bValue = swModel.Extension.SelectByID2("Extrude1", "SOLIDBODY", 0, 0, 0, True, 0, Nothing, 0)  
Set swWorkPieceBody = swModel.SelectionManager.GetSelectedObject6(1, -1)  
  
' Create temporary body by copying the selected body  
' Boolean operations consume bodies  
Set swWorkPieceBodyCopy = swWorkPieceBody.Copy  
  
' Clear the selection  
swModel.ClearSelection2 True  
  
' Collect the faces on the selected body  
lIdx = 0  
Set swFace = swWorkPieceBodyCopy.GetFirstFace  
Do While (Not (swFace Is Nothing))  
    ReDim Preserve aFaces(lIdx)  
    Set aFaces(lIdx) = swFace
```

```
lIdx = lIdx + 1
Set swFace = swFace.GetNextFace
Loop

' Create a sheet body from the faces
vFaces = aFaces
Set swWorkPieceFacesBody = swModeler.CreateSheetFromFaces(vFaces)

' Create a feature from the sheet body
Set aBodies(0) = swWorkPieceFacesBody
vBodies = aBodies
Set swPart = swModel
Set swFeature = swPart.CreateFeatureFromBody3(vBodies(0), False, swCreateFeatureBodyOpts_e.swCreateFeatureBodyCheck)

' Rebuild the model; a Surface-Imported feature should
' appear in the FeatureManager design tree after the rebuild
swModel.EditRebuild3

End Sub
```

Create Sheet Metal Costing Analysis Example (C#)

This example shows how to create a sheet metal Costing analysis.

```
//-----
// Preconditions:
// 1. Open:
//    C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\costing\sheet_metal_part.sldprt
// 2. Verify that the Costing templates exist by clicking Tools > Options >
//    System Options > File Locations and select Costing templates in
//    Show folders for in SOLIDWORKS Design. Click Cancel to close the dialog.
// 3. Add a reference to SolidWorks.Interop.sldcostingapi.dll.
// 4. Open the Immediate window.
// 5. Run the macro.
//
// Postconditions:
// 1. If prompted to save a temporary template with the costing data,
//    click Yes, browse to the folder where to save the temporary
//    template, type the name of the temporary template in File name,
//    and click Save.
// 2. Creates a sheet metal Costing analysis, which
//    might take one or more minutes to complete.
// 3. Examine the Immediate window.
//
// NOTE: Because the part is used elsewhere, do not
// save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using SolidWorks.Interop.sldcostingapi;
using System.Runtime.InteropServices;
using System.Diagnostics;
using System;

namespace SOLIDWORKSSheetMetalCostingCSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        ModelDocExtension swModelDocExt;
        CostManager swCosting;
        CostPart swCostingPart;
        CostBody swCostingBody;
        object swCostingModel;
        CostAnalysis swCostingAnalysis;
        CostAnalysisSheetMetal swCostingSheetMetal;
        string sheetmetalCostingTemplatePathName;
        string sheetmetalCostingReportTemplateFolderName;
        int nbrSheetmetalCostingTemplate;
        int nbrCommonCostingTemplate;
        object[] commonCostingTemplates;
        object[] sheetmetalCostingTemplates;
        int nbrCostingBodies;
        object[] costingBodies;
        string costingBodyName;
        int i;
        bool isSheetMetalBody;
        CostFeature swCostingFeat;
        CostFeature swCostingNextFeat;
        CostFeature swCostingSubFeat;
        CostFeature swCostingNextSubFeat;

        public void Main()
        {
            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelDocExt = swModel.Extension;

            // Get Costing templates names
            Debug.Print("Costing template folders:");
            sheetmetalCostingTemplatePathName = swApp.GetUserPreferenceStringValue((int)swUserPreferenceStringValue_e.swFileLocationsCo
            Debug.Print("    Name of Costing template folder: " + sheetmetalCostingTemplatePathName);
            sheetmetalCostingReportTemplateFolderName = swApp.GetUserPreferenceStringValue((int)swUserPreferenceStringValue_e.swFileLoc
            Debug.Print("    Name of Costing report template folder: " + sheetmetalCostingReportTemplateFolderName);
            Debug.Print("");

            // Get CostingManager
            swCosting = (CostManager)swModelDocExt.GetCostingManager();
            swCosting.WaitForUIUpdate();

            // Get the number of templates
            nbrSheetmetalCostingTemplate = swCosting.GetTemplateCount((int)swcCostingType_e.swcCostingType_SheetMetal);
            nbrCommonCostingTemplate = swCosting.GetTemplateCount((int)swcCostingType_e.swcCostingType_Common);

            // Get names of templates
            sheetmetalCostingTemplates = (object[])swCosting.GetTemplatePathnames((int)swcCostingType_e.swcCostingType_SheetMetal);
```

```

commonCostingTemplates = (object[])swCosting.GetTemplatePathnames((int)swcCostingType_e.swcCostingType_Common);

Array.Resize(ref sheetmetalCostingTemplates, nbrSheetmetalCostingTemplate + 1);
Array.Resize(ref commonCostingTemplates, nbrCommonCostingTemplate + 1);

Debug.Print("Costing templates:");

// Print names of templates to Immediate window
for (i = 0; i <= (nbrSheetmetalCostingTemplate - 1); i++)
{
    Debug.Print("    Name of sheet metal Costing template: " + sheetmetalCostingTemplates[i]);
}

Debug.Print("");

for (i = 0; i <= (nbrCommonCostingTemplate - 1); i++)
{
    Debug.Print("    Name of common Costing template: " + commonCostingTemplates[i]);
}

Debug.Print("");

// Get Costing part
swCostingModel = (object)swCosting.CostingModel;
swCostingPart = (CostPart)swCostingModel;

// Create common Costing analysis
swCostingAnalysis = (CostAnalysis)swCostingPart.CreateCostAnalysis("c:\\program files\\solidworks corp\\solidworks\\lang\\e

// Get common Costing analysis data
Debug.Print("Common Costing analysis data:");
Debug.Print("    Template name: " + swCostingAnalysis.CostingTemplateName);
Debug.Print("    Currency code: " + swCostingAnalysis.CurrencyCode);
Debug.Print("    Currency name: " + swCostingAnalysis.CurrencyName);
Debug.Print("    Currency separator: " + swCostingAnalysis.CurrencySeparator);
Debug.Print("    Total manufacturing cost: " + swCostingAnalysis.GetManufacturingCost());
Debug.Print("    Material costs: " + swCostingAnalysis.GetMaterialCost());
Debug.Print("    Setup cost: " + swCostingAnalysis.GetSetupCost());
Debug.Print("    Total cost to charge: " + swCostingAnalysis.GetTotalCostToCharge());
Debug.Print("    Total cost to manufacture: " + swCostingAnalysis.GetTotalCostToManufacture());
Debug.Print("    Lot size: " + swCostingAnalysis.LotSize);
Debug.Print("    Total quantity: " + swCostingAnalysis.TotalQuantity);
Debug.Print("");

// Get Costing bodies
nbrCostingBodies = swCostingPart.GetBodyCount();
if ((nbrCostingBodies > 0))
{
    Debug.Print("Costing bodies:");
    Debug.Print("    Number of Costing bodies: " + nbrCostingBodies);
    costingBodies = (object[])swCostingPart.GetBodies();
    for (i = 0; i <= (nbrCostingBodies - 1); i++)
    {
        swCostingBody = (CostBody)costingBodies[i];
        costingBodyName = (string)swCostingBody.GetName();
        Debug.Print("    Name of Costing body: " + costingBodyName);
        isSheetMetalBody = false;
        // Make sure body is sheet metal
        if ((swCostingBody.GetBodyType() == (int)swcBodyType_e.swcBodyType_SheetMetal))
        {
            isSheetMetalBody = true;
            // Determine analysis status of Costing body
            switch ((int)swCostingBody.BodyStatus)
            {
                case (int)swcBodyStatus_e.swcBodyStatus_NotAnalysed:
                    // Create Costing analysis
                    swCostingAnalysis = (CostAnalysis)swCostingBody.CreateCostAnalysis("c:\\program files\\solidworks corp\\
                    Debug.Print("    Creating sheet metal Costing analysis for: " + swCostingBody.GetName());
                    break;
                case (int)swcBodyStatus_e.swcBodyStatus_Analysed:
                    // Get Costing analysis
                    swCostingAnalysis = (CostAnalysis)swCostingBody.GetCostAnalysis();
                    Debug.Print("    Getting sheet metal Costing analysis for: " + swCostingBody.GetName());
                    break;
                case (int)swcBodyStatus_e.swcBodyStatus_Excluded:
                    // Body excluded from Costing analysis
                    Debug.Print("    Excluded from sheet metal Costing analysis: " + swCostingBody.GetName());
                    break;
                case (int)swcBodyStatus_e.swcBodyStatus_AssignedCustomCost:
                    // Body has an assigned custom Cost
                    Debug.Print("    Custom cost assigned: " + swCostingBody.GetName());
                    break;
            }
        }
        Debug.Print("");
    }

    if (!isSheetMetalBody )
    {
        Debug.Print("");
        Debug.Print("No sheet metal bodies in part! Exiting macro.");
        return;
    }
}
}

```

```

    }

    // Get sheet metal Costing Analysis data
    swCostingSheetMetal = (CostAnalysisSheetMetal)swCostingAnalysis.GetSpecificAnalysis();
    Debug.Print("Sheet metal Costing analysis: ");
    Debug.Print("    Current material: " + swCostingSheetMetal.CurrentMaterial);
    Debug.Print("    Current material class: " + swCostingSheetMetal.CurrentMaterialClass);
    Debug.Print("    Current thickness: " + swCostingSheetMetal.CurrentThickness);
    Debug.Print("    Material cost: " + swCostingSheetMetal.MaterialCost);
    Debug.Print("    Material cost from template: " + swCostingSheetMetal.MaterialCostFromTemplate);
    Debug.Print("    Type of blank size: " + swCostingSheetMetal.BlankSizeType);
    Debug.Print("    Blank area from model: " + swCostingSheetMetal.BlankAreaFromModel);
    Debug.Print("    Blank area, include any margins, from model: " + swCostingSheetMetal.FinalAreaFromModel);
    Debug.Print("");

    // Get Costing features
    Debug.Print("Costing features:");
    swCostingFeat = (CostFeature)swCostingAnalysis.GetFirstCostFeature();
    while ((swCostingFeat != null))
    {
        Debug.Print("    Feature: " + swCostingFeat.Name);
        Debug.Print("    Type: " + swCostingFeat.GetType());
        Debug.Print("    Setup related: " + swCostingFeat.IsSetup);
        Debug.Print("    Overridden: " + swCostingFeat.IsOverridden);
        Debug.Print("    Combined cost: " + swCostingFeat.CombinedCost);
        Debug.Print("    Combined time: " + swCostingFeat.CombinedTime);

        swCostingSubFeat = (CostFeature)swCostingFeat.GetFirstSubFeature();
        while ((swCostingSubFeat != null))
        {
            Debug.Print("    Subfeature: " + swCostingSubFeat.Name);
            Debug.Print("    Type: " + swCostingSubFeat.GetType());
            Debug.Print("    Setup related: " + swCostingSubFeat.IsSetup);
            Debug.Print("    Overridden: " + swCostingSubFeat.IsOverridden);
            Debug.Print("    Combined cost: " + swCostingSubFeat.CombinedCost);
            Debug.Print("    Combined time: " + swCostingSubFeat.CombinedTime);
            swCostingNextSubFeat = (CostFeature)swCostingSubFeat.GetNextFeature();
            swCostingSubFeat = null;
            swCostingSubFeat = (CostFeature)swCostingNextSubFeat;
            swCostingNextSubFeat = null;
        }
        swCostingNextFeat = swCostingFeat.GetNextFeature();
        swCostingFeat = null;
        swCostingFeat = (CostFeature)swCostingNextFeat;
        swCostingNextFeat = null;
    }
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>

public SldWorks swApp;
}
}

```

Create Sheet Metal Costing Analysis Example (VBA)

This example shows how to create a sheet metal Costing analysis.

```

'-----
' Preconditions:
' 1. Open:
'    C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\costing\sheet_metal_part.sldprt
' 2. Verify that the Costing templates exist by clicking Tools > Options >
'    System Options > File Locations and selecting Costing templates in
'    Show folders for in SOLIDWORKS Design. Click Cancel to close the dialog.
' 3. Add a reference to SldCosting API 20nn Type Library.
' 4. Open the Immediate window.
' 5. Run the macro.
'
' Postconditions:
' 1. If prompted to save a temporary template with the costing data,
'    click Yes, browse to the folder where to save the temporary
'    template, type the name of the temporary template in File name,
'    and click Save.
' 2. Creates a sheet metal analysis, which might
'    take one or more minutes to complete.
' 3. Examine the Immediate window.
'
' NOTE: Because the part is used elsewhere, do not
' save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swCosting As SldCostingAPI.CostManager
Dim swCostingPart As SldCostingAPI.CostPart
Dim swCostingBody As SldCostingAPI.CostBody
Dim swCostingModel As Object
Dim swCostingAnalysis As SldCostingAPI.CostAnalysis
Dim swCostingSheetMetal As SldCostingAPI.CostAnalysisSheetMetal
Dim sheetmetalCostingTemplatePathName As String
Dim sheetmetalCostingReportTemplateFolderName As String
Dim nbrSheetmetalCostingTemplate As Long
Dim nbrcommonCostingTemplate As Long
Dim commonCostingTemplates As Variant
Dim sheetmetalCostingTemplates As Variant
Dim nbrCostingBodies As Long
Dim costingBodies As Variant
Dim costingBodyName As String
Dim i As Long
Dim isSheetMetalBody As Boolean
Dim swCostingFeat As SldCostingAPI.CostFeature
Dim swCostingNextFeat As SldCostingAPI.CostFeature
Dim swCostingSubFeat As SldCostingAPI.CostFeature
Dim swCostingNextSubFeat As SldCostingAPI.CostFeature

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swModelDocExt = swModel.Extension

    ' Get Costing templates names
    Debug.Print ("Costing template folders:")
    sheetmetalCostingTemplatePathName = swApp.GetUserPreferenceStringValue(swUserPreferenceStringValue_e.swFileLocationsCostingTemplates)
    Debug.Print "    Name of Costing template folder: " & sheetmetalCostingTemplatePathName
    sheetmetalCostingReportTemplateFolderName =
    swApp.GetUserPreferenceStringValue(swUserPreferenceStringValue_e.swFileLocationsCostingReportTemplateFolder)
    Debug.Print "    Name of Costing report template folder: " & sheetmetalCostingReportTemplateFolderName
    Debug.Print ""

    ' Get CostingManager
    Set swCosting = swModelDocExt.GetCostingManager
    swCosting.WaitForUIUpdate

    ' Get the number of templates
    nbrSheetmetalCostingTemplate = swCosting.GetTemplateCount(swcCostingType_e.swcCostingType_SheetMetal)
    nbrcommonCostingTemplate = swCosting.GetTemplateCount(swcCostingType_e.swcCostingType_Common)

    ' Get names of templates
    sheetmetalCostingTemplates = swCosting.GetTemplatePathnames(swcCostingType_e.swcCostingType_SheetMetal)
    commonCostingTemplates = swCosting.GetTemplatePathnames(swcCostingType_e.swcCostingType_Common)

    ReDim Preserve sheetmetalCostingTemplates(nbrSheetmetalCostingTemplate)
    ReDim Preserve commonCostingTemplates(nbrcommonCostingTemplate)

    Debug.Print ("Costing templates:")

    ' Print names of templates to Immediate window
    For i = 0 To (UBound(sheetmetalCostingTemplates) - 1)
        Debug.Print "    Name of sheet metal Costing template: " & sheetmetalCostingTemplates(i)
    Next i

    Debug.Print ""

    For i = 0 To (UBound(commonCostingTemplates) - 1)
        Debug.Print "    Name of common Costing template: " & commonCostingTemplates(i)
    Next i

```

```

Debug.Print ""

' Get Costing part
Set swCostingModel = swCosting.CostingModel
Set swCostingPart = swCostingModel

' Create common Costing analysis
Set swCostingAnalysis = swCostingPart.CreateCostAnalysis("c:\program files\solidworks corp\solidworks\lang\english\costing
templates\multibodytemplate_default(englishstandard).sldctc")

' Get common Costing analysis data
Debug.Print ("Common Costing analysis data:")
Debug.Print ("    Template name: " & swCostingAnalysis.CostingTemplateName)
Debug.Print ("    Currency code: " & swCostingAnalysis.CurrencyCode)
Debug.Print ("    Currency name: " & swCostingAnalysis.CurrencyName)
Debug.Print ("    Currency separator: " & swCostingAnalysis.CurrencySeparator)
Debug.Print ("    Total manufacturing cost: " & swCostingAnalysis.GetManufacturingCost)
Debug.Print ("    Material costs: " & swCostingAnalysis.GetMaterialCost)
Debug.Print ("    Setup cost: " & swCostingAnalysis.GetSetupCost)
Debug.Print ("    Total cost to charge: " & swCostingAnalysis.GetTotalCostToCharge)
Debug.Print ("    Total cost to manufacture: " & swCostingAnalysis.GetTotalCostToManufacture)
Debug.Print ("    Lot size: " & swCostingAnalysis.LotSize)
Debug.Print ("    Total quantity: " & swCostingAnalysis.TotalQuantity)
Debug.Print ""

' Get Costing bodies
nbrCostingBodies = swCostingPart.GetBodyCount
If (nbrCostingBodies > 0) Then
    Debug.Print "Costing bodies:"
    Debug.Print ("    Number of Costing bodies: " & nbrCostingBodies)
    costingBodies = swCostingPart.GetBodies
    For i = 0 To (nbrCostingBodies - 1)
        Set swCostingBody = costingBodies(i)
        costingBodyName = swCostingBody.GetName
        Debug.Print ("    Name of Costing body: " & costingBodyName)
        isSheetMetalBody = False
        ' Make sure body is sheet metal
        If (swCostingBody.GetBodyType = swcBodyType_e.swcBodyType_SheetMetal) Then
            isSheetMetalBody = True
            ' Determine analysis status of Costing body
            Select Case swCostingBody.BodyStatus
                Case swcBodyStatus_e.swcBodyStatus_NotAnalysed
                    ' Create Costing analysis
                    Set swCostingAnalysis = swCostingBody.CreateCostAnalysis("c:\program files\solidworks
corp\solidworks\lang\english\costing templates\sheetmetaltemplate_default(englishstandard).sldcts")
                    Debug.Print ("    Creating sheet metal Costing analysis for: " & swCostingBody.GetName)
                Case swcBodyStatus_e.swcBodyStatus_Analysed
                    ' Get Costing analysis
                    Set swCostingAnalysis = swCostingBody.GetCostAnalysis
                    Debug.Print ("    Getting sheet metal Costing analysis for: " & swCostingBody.GetName)
                Case swcBodyStatus_e.swcBodyStatus_Excluded
                    ' Body excluded from Costing analysis
                    Debug.Print ("    Excluded from sheet metal Costing analysis: " & swCostingBody.GetName)
                Case swcBodyStatus_e.swcBodyStatus_AssignedCustomCost
                    ' Body has an assigned custom Cost
                    Debug.Print ("    Custom cost assigned: " & swCostingBody.GetName)
            End Select

            Debug.Print ""

        End If
    Next i

    If Not isSheetMetalBody Then
        Debug.Print ""
        Debug.Print "No sheet metal bodies in part! Exiting macro."
        Exit Sub
    End If
End If

' Get sheet metal Costing Analysis data
Set swCostingSheetMetal = swCostingAnalysis.GetSpecificAnalysis
Debug.Print "Sheet metal Costing analysis: "
Debug.Print ("    Current material: " & swCostingSheetMetal.CurrentMaterial)
Debug.Print ("    Current material class: " & swCostingSheetMetal.CurrentMaterialClass)
Debug.Print ("    Current thickness: " & swCostingSheetMetal.CurrentThickness)
Debug.Print ("    Material cost: " & swCostingSheetMetal.MaterialCost)
Debug.Print ("    Material cost from template: " & swCostingSheetMetal.MaterialCostFromTemplate)
Debug.Print ("    Type of blank size: " & swCostingSheetMetal.BlankSizeType)
Debug.Print ("    Blank area from model: " & swCostingSheetMetal.BlankAreaFromModel)
Debug.Print ("    Blank area, include any margins, from model: " & swCostingSheetMetal.FinalAreaFromModel)

Debug.Print ("")

' Get Costing features
Debug.Print "Costing features:"
Set swCostingFeat = swCostingAnalysis.GetFirstCostFeature
While Not swCostingFeat Is Nothing
    Debug.Print ("    Feature: " & swCostingFeat.Name)
    Debug.Print ("    Type: " & swCostingFeat.GetType)
    Debug.Print ("    Setup related: " & swCostingFeat.IsSetup)
    Debug.Print ("    Overridden: " & swCostingFeat.IsOverridden)
    Debug.Print ("    Combined cost: " & swCostingFeat.CombinedCost)
    Debug.Print ("    Combined time: " & swCostingFeat.CombinedTime)

    Set swCostingSubFeat = swCostingFeat.GetFirstSubFeature
    While Not swCostingSubFeat Is Nothing
        Debug.Print ("        Subfeature: " & swCostingSubFeat.Name)
        Debug.Print ("        Type: " & swCostingSubFeat.GetType)
        Debug.Print ("        Setup related: " & swCostingSubFeat.IsSetup)
        Debug.Print ("        Overridden: " & swCostingSubFeat.IsOverridden)
    End While
End While

```

```
        Debug.Print ("          Combined cost: " & swCostingSubFeat.CombinedCost)
        Debug.Print ("          Combined time: " & swCostingSubFeat.CombinedTime)
        Set swCostingNextSubFeat = swCostingSubFeat.GetNextFeature
        Set swCostingSubFeat = Nothing
        Set swCostingSubFeat = swCostingNextSubFeat
        Set swCostingNextSubFeat = Nothing
    Wend
    Set swCostingNextFeat = swCostingFeat.GetNextFeature
    Set swCostingFeat = Nothing
    Set swCostingFeat = swCostingNextFeat
    Set swCostingNextFeat = Nothing
Wend
```

End Sub

Create Sheet Metal Costing Analysis Example (VB.NET)

This example shows how to create a sheet metal Costing analysis.

```

'-----
' Preconditions:
' 1. Open:
'    C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\costing\sheet_metal_part.sldprt
' 2. Verify that the Costing templates exist by clicking Tools > Options >
'    System Options > File Locations and selecting Costing templates in
'    Show folders for in SOLIDWORKS Design. Click Cancel to close the dialog.
' 3. Add a reference to SolidWorks.Interop.sldcostingapi.dll.
' 4. Open the Immediate window.
' 5. Run the macro.
'
' Postconditions:
' 1. If prompted to save a temporary template with the costing data,
'    click Yes, browse to the folder where to save the temporary
'    template, type the name of the temporary template in File name,
'    and click Save.
' 2. Creates a sheet metal analysis, which might
'    take one or more minutes to complete.
' 3. Examine the Immediate window.
'
' NOTE: Because the part is used elsewhere, do not
' save changes.
'-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports SolidWorks.Interop.sldcostingapi
Imports System.Runtime.InteropServices
Imports System.Diagnostics
Imports System

```

Partial Class SolidWorksMacro

```

    Dim swModel As ModelDoc2
    Dim swModelDocExt As ModelDocExtension
    Dim swCosting As CostManager
    Dim swCostingPart As CostPart
    Dim swCostingBody As CostBody
    Dim swCostingModel As Object
    Dim swCostingAnalysis As CostAnalysis
    Dim swCostingSheetMetal As CostAnalysisSheetMetal
    Dim sheetmetalCostingTemplatePathName As String
    Dim sheetmetalCostingReportTemplateFolderName As String
    Dim nbrSheetmetalCostingTemplate As Integer
    Dim nbrcommonCostingTemplate As Integer
    Dim commonCostingTemplates As Object
    Dim sheetmetalCostingTemplates As Object
    Dim nbrCostingBodies As Integer
    Dim costingBodies As Object
    Dim costingBodyName As String
    Dim i As Integer
    Dim isSheetMetalBody As Boolean
    Dim swCostingFeat As CostFeature
    Dim swCostingNextFeat As CostFeature
    Dim swCostingSubFeat As CostFeature
    Dim swCostingNextSubFeat As CostFeature

```

Sub main()

```

    swModel = swApp.ActiveDoc
    swModelDocExt = swModel.Extension

```

```

    ' Get Costing templates names
    Debug.Print("Costing template folders:")
    sheetmetalCostingTemplatePathName = swApp.GetUserPreferenceStringValue(swUserPreferenceStringValue_e.swFileLocationsCostingTemp
    Debug.Print("    Name of Costing template folder: " & sheetmetalCostingTemplatePathName)
    sheetmetalCostingReportTemplateFolderName = swApp.GetUserPreferenceStringValue(swUserPreferenceStringValue_e.swFileLocationsCos
    Debug.Print("    Name of Costing report template folder: " & sheetmetalCostingReportTemplateFolderName)
    Debug.Print("")

```

```

    ' Get CostingManager
    swCosting = swModelDocExt.GetCostingManager
    swCosting.WaitForUIUpdate()

```

```

    ' Get the number of templates
    nbrSheetmetalCostingTemplate = swCosting.GetTemplateCount(swcCostingType_e.swcCostingType_SheetMetal)
    nbrcommonCostingTemplate = swCosting.GetTemplateCount(swcCostingType_e.swcCostingType_Common)

```

```

    ' Get names of templates
    sheetmetalCostingTemplates = swCosting.GetTemplatePathnames(swcCostingType_e.swcCostingType_SheetMetal)
    commonCostingTemplates = swCosting.GetTemplatePathnames(swcCostingType_e.swcCostingType_Common)

```

```

    ReDim Preserve sheetmetalCostingTemplates(nbrSheetmetalCostingTemplate)
    ReDim Preserve commonCostingTemplates(nbrcommonCostingTemplate)

```

```

Debug.Print("Costing templates:")

' Print names of templates to Immediate window
For i = 0 To (UBound(sheetmetalCostingTemplates) - 1)
    Debug.Print("    Name of sheet metal Costing template: " & sheetmetalCostingTemplates(i))
Next i

Debug.Print("")

For i = 0 To (UBound(commonCostingTemplates) - 1)
    Debug.Print("    Name of common Costing template: " & commonCostingTemplates(i))
Next i

Debug.Print("")

' Get Costing part
swCostingModel = swCosting.CostingModel
swCostingPart = swCostingModel

' Create common Costing analysis
swCostingAnalysis = swCostingPart.CreateCostAnalysis("c:\program files\solidworks corp\solidworks\lang\english\costing template

' Get common Costing analysis data
Debug.Print("Common Costing analysis data:")
Debug.Print("    Template name: " & swCostingAnalysis.CostingTemplateName)
Debug.Print("    Currency code: " & swCostingAnalysis.CurrencyCode)
Debug.Print("    Currency name: " & swCostingAnalysis.CurrencyName)
Debug.Print("    Currency separator: " & swCostingAnalysis.CurrencySeparator)
Debug.Print("    Total manufacturing cost: " & swCostingAnalysis.GetManufacturingCost)
Debug.Print("    Material costs: " & swCostingAnalysis.GetMaterialCost)
Debug.Print("    Setup cost: " & swCostingAnalysis.GetSetupCost)
Debug.Print("    Total cost to charge: " & swCostingAnalysis.GetTotalCostToCharge)
Debug.Print("    Total cost to manufacture: " & swCostingAnalysis.GetTotalCostToManufacture)
Debug.Print("    Lot size: " & swCostingAnalysis.LotSize)
Debug.Print("    Total quantity: " & swCostingAnalysis.TotalQuantity)
Debug.Print("")

' Get Costing bodies
nbrCostingBodies = swCostingPart.GetBodyCount
If (nbrCostingBodies > 0) Then
    Debug.Print("Costing bodies:")
    Debug.Print("    Number of Costing bodies: " & nbrCostingBodies)
    costingBodies = swCostingPart.GetBodies
    For i = 0 To (nbrCostingBodies - 1)
        swCostingBody = costingBodies(i)
        costingBodyName = swCostingBody.GetName
        Debug.Print("    Name of Costing body: " & costingBodyName)
        isSheetMetalBody = False
        ' Make sure body is sheet metal
        If (swCostingBody.GetBodyType = swcBodyType_e.swcBodyType_SheetMetal) Then
            ' Determine analysis status of Costing body
            isSheetMetalBody = True
            Select Case swCostingBody.BodyStatus
                Case swcBodyStatus_e.swcBodyStatus_NotAnalysed
                    ' Create Costing analysis
                    swCostingAnalysis = swCostingBody.CreateCostAnalysis("c:\program files\solidworks corp\solidworks\lang\engl
                    Debug.Print("    Creating sheet metal Costing analysis for: " & swCostingBody.GetName)
                Case swcBodyStatus_e.swcBodyStatus_Analysed
                    ' Get Costing analysis
                    swCostingAnalysis = swCostingBody.GetCostAnalysis
                    Debug.Print("    Getting sheet metal Costing analysis for: " & swCostingBody.GetName)
                Case swcBodyStatus_e.swcBodyStatus_Excluded
                    ' Body excluded from Costing analysis
                    Debug.Print("    Excluded from sheet metal Costing analysis: " & swCostingBody.GetName)
                Case swcBodyStatus_e.swcBodyStatus_AssignedCustomCost
                    ' Body has an assigned custom Cost
                    Debug.Print("    Custom cost assigned: " & swCostingBody.GetName)
            End Select
        End If
        Debug.Print("")
    Next i
End If

If Not isSheetMetalBody Then
    Debug.Print("")
    Debug.Print("No sheet metal bodies in part! Exiting macro.")
    Exit Sub
End If

End If

' Get sheet metal Costing Analysis data
swCostingSheetMetal = swCostingAnalysis.GetSpecificAnalysis
Debug.Print("Sheet metal Costing analysis: ")
Debug.Print("    Current material: " & swCostingSheetMetal.CurrentMaterial)
Debug.Print("    Current material class: " & swCostingSheetMetal.CurrentMaterialClass)
Debug.Print("    Current thickness: " & swCostingSheetMetal.CurrentThickness)
Debug.Print("    Material cost: " & swCostingSheetMetal.MaterialCost)
Debug.Print("    Material cost from template: " & swCostingSheetMetal.MaterialCostFromTemplate)
Debug.Print("    Type of blank size: " & swCostingSheetMetal.BlankSizeType)
Debug.Print("    Blank area from model: " & swCostingSheetMetal.BlankAreaFromModel)
Debug.Print("    Blank area, include any margins, from model: " & swCostingSheetMetal.FinalAreaFromModel)

```

```

Debug.Print("")

' Get Costing features
Debug.Print("Costing features:")
swCostingFeat = swCostingAnalysis.GetFirstCostFeature
While Not swCostingFeat Is Nothing
    Debug.Print("    Feature: " & swCostingFeat.Name)
    Debug.Print("    Type: " & swCostingFeat.GetType)
    Debug.Print("    Setup related: " & swCostingFeat.IsSetup)
    Debug.Print("    Overridden: " & swCostingFeat.IsOverridden)
    Debug.Print("    Combined cost: " & swCostingFeat.CombinedCost)
    Debug.Print("    Combined time: " & swCostingFeat.CombinedTime)

    swCostingSubFeat = swCostingFeat.GetFirstSubFeature
    While Not swCostingSubFeat Is Nothing
        Debug.Print("        Subfeature: " & swCostingSubFeat.Name)
        Debug.Print("        Type: " & swCostingSubFeat.GetType)
        Debug.Print("        Setup related: " & swCostingSubFeat.IsSetup)
        Debug.Print("        Overridden: " & swCostingSubFeat.IsOverridden)
        Debug.Print("        Combined cost: " & swCostingSubFeat.CombinedCost)
        Debug.Print("        Combined time: " & swCostingSubFeat.CombinedTime)
        swCostingNextSubFeat = swCostingSubFeat.GetNextFeature
        swCostingSubFeat = Nothing
        swCostingSubFeat = swCostingNextSubFeat
        swCostingNextSubFeat = Nothing
    End While
    swCostingNextFeat = swCostingFeat.GetNextFeature
    swCostingFeat = Nothing
    swCostingFeat = swCostingNextFeat
    swCostingNextFeat = Nothing
End While

End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

```

End Class

Create Sheet Metal Gusset Example (C#)

This example shows how to create a sheet metal gusset feature.

```
// -----
// Preconditions:
// 1. Open public_documents\samples\tutorial\api\SMGussetAPI.sldprt.
// 2. Open an Immediate window.
//
// Postconditions:
// 1. Inserts Sheet Metal Gusset1.
// 2. Press F5 and observe the modified gusset.
// 3. Inspect the Immediate window for the flatten settings of the gusset.
// 4. Expand Flat-Pattern in the FeatureManager design tree, right-click
//    Flat-Pattern(1), and click Unsuppress.
// 5. Observe the center mark and profile of the gusset in its
//    flattened state.
//
// NOTE: Because the model is used elsewhere, do not save changes.
// -----
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows;
using System.Windows.Forms;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Diagnostics;

namespace CreateSheetMetalGusset_CSharp
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 Part;
            Feature myFeature;
            Feature swFeat;
            SMGussetFeatureData swFeatData;
            bool boolstatus;

            Part = (ModelDoc2)swApp.ActiveDoc;

            boolstatus = Part.Extension.SelectByID("", "FACE", -0.0538403893476698, 0.0036701308754914, 0.05530817474488,
false, 0, null, 0);
            boolstatus = Part.Extension.SelectByID("", "FACE", -0.0177780871801474, -0.0307393226379986, 0.0341128529187245,
true, 0, null, 0);

            swFeatData = (SMGussetFeatureData)Part.FeatureManager.CreateDefinition((int)swFeatureNameID_e.swFmSMGusset);
            swFeatData.UseOffset = true;
            swFeatData.OffsetDistance = 0.05;
            swFeatData.FlipOffset = false;
            swFeatData.ProfileDimensionScheme =
(int)swSheetMetalGussetProfileDimType_e.swSheetMetalGussetProfileDimType_IndentDepth;
            swFeatData.IndentDepth = 0.01;
            swFeatData.ProfileLengthDim = 0.0;
            swFeatData.UseAngleDimForProfile = false;
            swFeatData.ProfileHeightDim = 0.0;
            swFeatData.ProfileAngleDim = 0.0;
```

```

swFeatData.FlipDimSides = false;
swFeatData.IndentWidth = 0.01;
swFeatData.GussetThickness = 0.003;
swFeatData.DraftAngle = 3 * 0.0175;
swFeatData.DraftSideFaces = true;
swFeatData.FilletInnerCorners = true;
swFeatData.InnerCornerFilletRadius = 0.002;
swFeatData.FilletOuterCorners = true;
swFeatData.OuterCornerFilletRadius = 0.001;
swFeatData.GussetType = (int)swSheetMetalRibGussetType_e.swSheetMetalRibGussetType_Rounded;
swFeatData.FilletGussetEdges = false;
swFeatData.EdgeFilletRadius = 0.0;
swFeatData.OverrideDocSettings = true;
swFeatData.ShowProfile = true;
swFeatData.ShowCenter = true;

myFeature = Part.FeatureManager.CreateFeature(swFeatData);
Part.ClearSelection2(true);

System.Diagnostics.Debugger.Break();

// Modify type, draft, and outer corner fillet properties of the gusset
boolstatus = Part.Extension.SelectByID2("Sheet Metal Gusset1", "BODYFEATURE", 0, 0, 0, false, 0, null, 0);
swFeat = (Feature)((SelectionMgr)Part.SelectionManager).GetSelectedObject6(1, -1);

swFeatData = (SMGussetFeatureData)swFeat.GetDefinition();
swFeatData.AccessSelections(Part, null);
swFeatData.GussetType = (int)swSheetMetalRibGussetType_e.swSheetMetalRibGussetType_Flat;
swFeatData.DraftSideFaces = false;
swFeatData.FilletOuterCorners = false;
Debug.Print("Sheet Metal Gusset1 Flatten Settings");
Debug.Print(" Override document flat pattern properties? " + swFeatData.OverrideDocSettings);
Debug.Print(" Show center marks of the gusset in its flattened state? " + swFeatData.ShowCenter);
Debug.Print(" Show profile of the gusset in its flattened state? " + swFeatData.ShowProfile);

swFeat.ModifyDefinition(swFeatData, Part, null);
swFeatData.ReleaseSelectionAccess();
}

// The SldWorks swApp variable is pre-assigned for you.
public SldWorks swApp;

}
}

```

Create Sheet Metal Gusset Example (VB.NET)

This example shows how to create a sheet metal gusset feature.

```
' -----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\SMGussetAPI.sldprt.
' 2. Open an Immediate window.
'
' Postconditions:
' 1. Inserts Sheet Metal Gusset1.
' 2. Press F5 and observe the modified gusset.
' 3. Inspect the Immediate window for the flatten settings of the gusset.
' 4. Expand Flat-Pattern in the FeatureManager design tree, right-click
'    Flat-Pattern(1), and click Unsuppress.
' 5. Observe the center mark and profile of the gusset in its
'    flattened state.
'
' NOTE: Because the model is used elsewhere, do not save changes.
' -----
```

```
Imports SolidWorks.Interop.sldworks
```

```
Imports SolidWorks.Interop.swconst
```

```
Imports System.Runtime.InteropServices
```

```
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Public Sub main()
```

```
        Dim Part As ModelDoc2
```

```
        Dim myFeature As Feature
```

```
        Dim swFeat As Feature
```

```
        Dim swFeatData As SMGussetFeatureData
```

```
        Dim boolstatus As Boolean
```

```
        Part = swApp.ActiveDoc
```

```
        boolstatus = Part.Extension.SelectByID2("", "FACE", -0.0538403893476698, 0.0036701308754914, 0.05530817474488, False, 0,
Nothing, 0)
```

```
        boolstatus = Part.Extension.SelectByID2("", "FACE", -0.0177780871801474, -0.0307393226379986, 0.0341128529187245, True, 0,
Nothing, 0)
```

```
        swFeatData = Part.FeatureManager.CreateDefinition(swFeatureNameID_e.swFmSMGusset)
```

```
        swFeatData.UseOffset = True
```

```
        swFeatData.OffsetDistance = 0.05
```

```
        swFeatData.FlipOffset = False
```

```
        swFeatData.ProfileDimensionScheme = swSheetMetalGussetProfileDimType_e.swSheetMetalGussetProfileDimType_IndentDepth
```

```
        swFeatData.IndentDepth = 0.01
```

```
        swFeatData.ProfileLengthDim = 0#
```

```
        swFeatData.UseAngleDimForProfile = False
```

```
        swFeatData.ProfileHeightDim = 0#
```

```
        swFeatData.ProfileAngleDim = 0#
```

```
        swFeatData.FlipDimSides = False
```

```
        swFeatData.IndentWidth = 0.01
```

```
        swFeatData.GussetThickness = 0.003
```

```
        swFeatData.DraftAngle = 3 * 0.0175
```

```
        swFeatData.DraftSideFaces = True
```

```
        swFeatData.FilletInnerCorners = True
```

```
        swFeatData.InnerCornerFilletRadius = 0.002
```

```
        swFeatData.FilletOuterCorners = True
```

```
        swFeatData.OuterCornerFilletRadius = 0.001
```

```
        swFeatData.GussetType = swSheetMetalRibGussetType_e.swSheetMetalRibGussetType_Rounded
```

```

swFeatData.FilletGussetEdges = False
swFeatData.EdgeFilletRadius = 0#
swFeatData.OverrideDocSettings = True
swFeatData.ShowProfile = True
swFeatData.ShowCenter = True

myFeature = Part.FeatureManager.CreateFeature(swFeatData)
Part.ClearSelection2(True)

```

Stop

```

'Modify type, draft, and outer corner fillet properties of the gusset
boolstatus = Part.Extension.SelectByID2("Sheet Metal Gusset1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
swFeat = Part.SelectionManager.GetSelectedObject6(1, -1)

```

```

swFeatData = swFeat.GetDefinition
swFeatData.AccessSelections(Part, Nothing)
swFeatData.GussetType = swSheetMetalRibGussetType_e.swSheetMetalRibGussetType_Flat
swFeatData.DraftSideFaces = False
swFeatData.FilletOuterCorners = False
Debug.Print("Sheet Metal Gusset1 Flatten Settings")
Debug.Print(" Override document flat pattern properties? " & swFeatData.OverrideDocSettings)
Debug.Print(" Show center marks of the gusset in its flattened state? " & swFeatData.ShowCenter)
Debug.Print(" Show profile of the gusset in its flattened state? " & swFeatData.ShowProfile)

swFeat.ModifyDefinition(swFeatData, Part, Nothing)
swFeatData.ReleaseSelectionAccess

```

End Sub

```

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>

```

Public swApp As SldWorks

End Class

Create Shell Feature Example (C#)

This example shows how to create a shell feature.

```
//-----
// Preconditions:
// 1. Verify that the specified model document exists.
// 2. Open an Immediate window.
//
// Postconditions:
// 1. Selects a face to remove from the model to create the shell.
// 2. Creates Shell1.
// 3. Inspect the Immediate window, graphics area, and
//    FeatureManager design tree.
//
// NOTE: Because the model is used elsewhere, do not save changes.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;

namespace CreateShellFeature_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        SelectionMgr swSelMgr;
        SelectData swSelData;
        Feature swFeat;
        ShellFeatureData swShell;
        object[] vFaceRemArr;
        object vFaceRem;
        Face2 swFaceRem;
        object[] vMultiFaceArr;
        object vMultiFace;
        Face2 swMultiFace;
        Entity swEnt;
        int i;
        bool bRet;
        int longstatus;
        int longwarnings;

        public void Main()
        {
            swModel = (ModelDoc2)swApp.OpenDoc6("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS
            2018\\samples\\tutorial\\api\\block20.sldprt", 1, 0, "", ref longstatus, ref longwarnings);
            swApp.ActivateDoc2("block20", false, ref longstatus);
            swModel = (ModelDoc2)swApp.ActiveDoc;

            bRet = swModel.Extension.SelectByID("", "FACE", -0.0150558029249623, 0.03962399999999466, -0.018063862472502, false, 1, null, 0);
            swModel.InsertFeatureShell(0.00254, false);

            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swSelData = swSelMgr.CreateSelectData();
            swFeat = (Feature)swSelMgr.GetSelectedObject6(1, -1);
            swShell = (ShellFeatureData)swFeat.GetDefinition();

            // Get shell data
            Debug.Print("File = " + swModel.GetPathName());
            Debug.Print(" " + swFeat.Name);
            Debug.Print(" Direction: " + swShell.Direction);
            Debug.Print(" Thickness: " + swShell.Thickness * 1000.0 + " mm");
            Debug.Print(" Count of faces removed: " + swShell.FacesRemovedCount);
            Debug.Print(" Count of faces with alternative thicknesses: " + swShell.GetMultipleThicknessFacesCount());

            bRet = swShell.AccessSelections(swModel, null);
            swModel.ClearSelection2(true);

            vFaceRemArr = (object[])swShell.FacesRemoved;

            for (i = 0; i < vFaceRemArr.GetLength(0); i++) {
                vFaceRem = vFaceRemArr[i];
                swFaceRem = (Face2)vFaceRem;
                swEnt = (Entity)swFaceRem;

                bRet = swEnt.Select4(true, swSelData);
            }

            swModel.ClearSelection2(true);
            vMultiFaceArr = (object[])swShell.MultipleThicknessFaces;
```

```

foreach (object vMultiFace_loopVariable in vMultiFaceArr) {
vMultiFace = vMultiFace_loopVariable;
swMultiFace = (Face2)vMultiFace;
swEnt = (Entity)swMultiFace;

Debug.Print("    Alternative thickness in mm at face (" + i + "): " + swShell.GetMultipleThicknessAtIndex(i) * 1000.0);
i = i + 1;

bRet = swEnt.Select4(true, swSelData);
}

swModel.ClearSelection2(true);
swShell.ReleaseSelectionAccess();

}

    /// <summary>
    /// The SldWorks swApp variable is pre-assigned for you.
    /// </summary>

    public SldWorks swApp;

}

}

```

Create Shell Feature Example (VB.NET)

This example shows how to create a shell feature..

```

'-----
' Preconditions:
' 1. Verify that the specified model document exists.
' 2. Open an Immediate window.
'
' Postconditions:
' 1. Selects a face to remove from the model to create the shell.
' 2. Creates Shell1.
' 3. Inspect the Immediate window, graphics area, and
'    FeatureManager design tree.
'
' NOTE: Because the model is used elsewhere, do not save changes.
'-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

```

Partial Class SolidWorksMacro

```

    Dim swModel As ModelDoc2
    Dim swSelMgr As SelectionMgr
    Dim swSelData As SelectData
    Dim swFeat As Feature
    Dim swShell As ShellFeatureData
    Dim vFaceRemArr As Object
    Dim vFaceRem As Object
    Dim swFaceRem As Face2
    Dim vMultiFaceArr As Object
    Dim vMultiFace As Object
    Dim swMultiFace As Face2
    Dim swEnt As Entity
    Dim i As Integer
    Dim bRet As Boolean
    Dim longstatus As Integer, longwarnings As Integer

    Sub main()

        swModel = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2018\samples\tutorial\api\block20.sldprt", 1, 0, "", longstatus, longwarnings)
        swApp.ActivateDoc2("block20", False, longstatus)
        swModel = swApp.ActiveDoc

        bRet = swModel.Extension.SelectByID2("", "FACE", -0.0150558029249623, 0.0396239999999466, -0.018063862472502, False, 1, NotF
        swModel.InsertFeatureShell(0.00254, False)

        swSelMgr = swModel.SelectionManager
        swSelData = swSelMgr.CreateSelectData
        swFeat = swSelMgr.GetSelectedObject6(1, -1)
        swShell = swFeat.GetDefinition

        ' Get shell data
        Debug.Print("File = " & swModel.GetPathName)
        Debug.Print(" " & swFeat.Name)
        Debug.Print(" Direction: " & swShell.Direction)
        Debug.Print(" Thickness: " & swShell.Thickness * 1000.0# & " mm")
        Debug.Print(" Count of faces removed: " & swShell.FacesRemovedCount)
        Debug.Print(" Count of faces with alternative thicknesses: " & swShell.GetMultipleThicknessFacesCount)

        bRet = swShell.AccessSelections(swModel, Nothing)
        swModel.ClearSelection2(True)

        vFaceRemArr = swShell.FacesRemoved

        For Each vFaceRem In vFaceRemArr
            swFaceRem = vFaceRem
            swEnt = swFaceRem

            bRet = swEnt.Select4(True, swSelData)
        Next

        swModel.ClearSelection2(True)
        vMultiFaceArr = swShell.MultipleThicknessFaces

        For Each vMultiFace In vMultiFaceArr
            swMultiFace = vMultiFace
            swEnt = swMultiFace

            Debug.Print(" Alternative thickness in mm at face (" & i & "): " & swShell.GetMultipleThicknessAtIndex(i) * 1000.0#)
            i = i + 1
        Next
    End Sub

```

```
        bRet = swEnt.Select4(True, swSelData)
    Next

    swModel.ClearSelection2(True)
    swShell.ReleaseSelectionAccess()
```

```
End Sub
```

```
''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
```

```
End Class
```

Create Simulation Gravity Feature Example (C#)

This example shows how to create a simulation gravity feature in a motion study.

```
//-----
// Preconditions:
// 1. Verify that the specified model document exists.
// 2. In SOLIDWORKS Design, verify that View > MotionManager is selected.
// 3. In the IDE, right-click the project name in the Project Explorer,
//    click Add Reference, browse to install_dir\api\redist, and select
//    SolidWorks.Interop.swmotionstudy.dll.
// 4. Click OK.
// 5. Open an Immediate window.
//
// Postconditions:
// 1. Adds Gravity to the MotionManager design tree on the Motion Study 1 tab.
// 2. Inspect the Immediate window.
//
// NOTE: Because the model is used elsewhere,
// do not save changes when closing it.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using SolidWorks.Interop.swmotionstudy;
using System.Runtime.InteropServices;

namespace CreateSimGravFeat_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        ModelDocExtension swModelDocExt;
        MotionStudyManager swMotionMgr;
        MotionStudy swMotionStudy1;
        SimulationGravityFeatureData swGravityFeat;
        Feature swFeat;
        int longstatus;
        int longwarnings;

        public void Main()
        {
            swModel = swApp.OpenDoc6("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS
2018\\samples\\tutorial\\api\\beam_boltconnection.sldasm", 2, 0, "", ref longstatus, ref longwarnings);
            swApp.ActivateDoc2("beam_boltconnection", false, ref longstatus);
            swModel = (ModelDoc2)swApp.ActiveDoc;

            swModelDocExt = swModel.Extension;

            swMotionMgr = (MotionStudyManager)swModelDocExt.GetMotionStudyManager();
            swMotionStudy1 = swMotionMgr.GetMotionStudy("Motion Study 1");

            if ((swMotionStudy1 == null))
            {
                Debug.Print("Motion Study 1 is not available");
            }

            swMotionStudy1.StudyType = (int)swMotionStudyType_e.swMotionStudyTypePhysicalSimulation;

            // Activate Motion Study 1
            swMotionStudy1.Activate();

            // Define feature data
            swGravityFeat = (SimulationGravityFeatureData)swMotionStudy1.CreateDefinition((int)swFeatureNameID_e.swFmAEMGravity);
            if (swGravityFeat == null)
            {
                Debug.Print("Creation of feature data object failed");
            }
            swGravityFeat.ReverseDirection = false;
            swGravityFeat.Axis = 0;
            swGravityFeat.Strength = 12;

            // Create feature
            swFeat = (Feature)swMotionStudy1.CreateFeature(swGravityFeat);

            if (swFeat == null)
            {
                Debug.Print("Creation of feature failed");
            }
            else
            {
                Debug.Print("Name of the feature added: " + swFeat.Name);
            }
        }
    }
}
```

```
    }  
}  
  
/// <summary>  
/// The SldWorks swApp variable is pre-assigned for you.  
/// </summary>  
public SldWorks swApp;  
}  
}
```

Create Simulation Gravity Feature Example (VBA)

This example shows how to create a simulation gravity feature in a motion study.

```

'-----
' Preconditions:
' 1. Verify that the specified model document exists.
' 2. In SOLIDWORKS Design, verify that View > MotionManager is selected.
' 3. In the IDE, click Tools > References and select
'    SOLIDWORKS version MotionStudy Type Library.
' 4. Click OK.
' 5. Open an Immediate window.
'
' Postconditions:
' 1. Adds Gravity to the MotionManager design tree on the Motion Study 1 tab.
' 2. Inspect the Immediate window.
'
' NOTE: Because the model is used elsewhere,
' do not save changes when closing it.
'-----

```

```

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swMotionMgr As SwMotionStudy.MotionStudyManager
Dim swMotionStudy1 As SwMotionStudy.MotionStudy
Dim swGravityFeat As SldWorks.SimulationGravityFeatureData
Dim swFeat As SldWorks.Feature
Dim lngstatus As Long, lngwarnings As Long
Option Explicit
Sub main()

    Set swApp = Application.SldWorks

    Set swModel = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2018\samples\tutorial\api\beam_boltconnection.sldasm", 2, 0, "", lngstatus, lngwarnings)
    swApp.ActivateDoc2 "beam_boltconnection", False, lngstatus
    Set swModel = swApp.ActiveDoc

    Set swModelDocExt = swModel.Extension

    Set swMotionMgr = swModelDocExt.GetMotionStudyManager()
    Set swMotionStudy1 = swMotionMgr.GetMotionStudy("Motion Study 1")
    If (swMotionStudy1 Is Nothing) Then
        Debug.Print "Motion Study 1 is not available"
    End If

    swMotionStudy1.StudyType = swMotionStudyType_e.swMotionStudyTypePhysicalSimulation
    ' Activate Motion Study 1
    swMotionStudy1.Activate
    ' Define feature data
    Set swGravityFeat = swMotionStudy1.CreateDefinition(swFmAEMGravity)
    If swGravityFeat Is Nothing Then
        Debug.Print "Creation of feature data object failed"
    End If

    swGravityFeat.ReverseDirection = False
    swGravityFeat.Axis = 0
    swGravityFeat.Strength = 12
    ' Create feature
    Set swFeat = swMotionStudy1.CreateFeature(swGravityFeat)

    If swFeat Is Nothing Then
        Debug.Print "Creation of feature failed"
    Else
        Debug.Print "Name of the feature added: " & swFeat.Name
    End If

End Sub

```

Create Simulation Gravity Feature Example (VB.NET)

This example shows how to create a simulation gravity feature in a motion study.

```

'-----
' Preconditions:
' 1. Verify that the specified model document exists.
' 2. In SOLIDWORKS Design, verify that View > MotionManager is selected.
' 3. In the IDE, right-click the project name in the Project Explorer,
'    click Add Reference, browse to install_dir\api\redist, and select
'    SolidWorks.Interop.swmotionstudy.dll.
' 4. Click OK.
' 5. Open an Immediate window.
'
' Postconditions:
' 1. Adds Gravity to the MotionManager design tree on the Motion Study 1 tab.
' 2. Inspect the Immediate window.
'
' NOTE: Because the model is used elsewhere,
' do not save changes when closing it.
'-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports SolidWorks.Interop.swmotionstudy
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

```

Partial Class SolidWorksMacro

```

    Dim swModel As ModelDoc2
    Dim swModelDocExt As ModelDocExtension
    Dim swMotionMgr As MotionStudyManager
    Dim swMotionStudy1 As MotionStudy
    Dim swGravityFeat As SimulationGravityFeatureData
    Dim boolstatus As Boolean
    Dim swFeat As Feature
    Dim longstatus As Integer, longwarnings As Integer

    Sub main()

        swModel = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2018\samples\tutorial\api\beam_boltconnection.sldasm", 2, 0, "", longstatus, longwarnings)
        swApp.ActivateDoc2("beam_boltconnection", False, longstatus)
        swModel = swApp.ActiveDoc

        swModelDocExt = swModel.Extension

        swMotionMgr = swModelDocExt.GetMotionStudyManager()
        swMotionStudy1 = swMotionMgr.GetMotionStudy("Motion Study 1")

        If (swMotionStudy1 Is Nothing) Then
            Debug.Print("Motion Study 1 is not available")
        End If

        swMotionStudy1.StudyType = swMotionStudyType_e.swMotionStudyTypePhysicalSimulation

        ' Activate Motion Study 1
        swMotionStudy1.Activate()

        ' Define feature data
        swGravityFeat = swMotionStudy1.CreateDefinition(swFeatureNameID_e.swFmAEMGravity)
        If swGravityFeat Is Nothing Then
            Debug.Print("Creation of feature data object failed")
        End If
        swGravityFeat.ReverseDirection = False
        swGravityFeat.Axis = 0
        swGravityFeat.Strength = 12

        ' Create feature
        swFeat = swMotionStudy1.CreateFeature(swGravityFeat)

        If swFeat Is Nothing Then
            Debug.Print("Creation of feature failed")
        Else
            Debug.Print("Name of the feature added: " & swFeat.Name)
        End If

    End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

```

End Class

Create Sketch Entities with Visual Properties Example (VBA)

This example shows how to create a drawing and add some sketch entities and change the visual properties of a couple of sketch entities.

```
'-----
' Preconditions: None.
'
' Postconditions:
' 1. Opens a new drawing and creates and activates a layer.
' 2. Inserts some sketch entities.
' 3. Examines the drawing, then press F5.
' 4. Changes the visual properties of a couple of sketch entities,
'    which overrides the visual properties of the layer.
' 5. Repaints the graphics area.
' 6. Examines the drawing.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim pDrawing As SldWorks.DrawingDoc
Dim pLayerMgr As SldWorks.LayerMgr
Dim pSketchSegment1, pSketchSegment2, pSketchSegment3 As Object
Dim pSketchSegment4, pSketchSegment5, pSketchPoint1, pSketchPoint2 As Object
Dim res As Long

Sub main()

    Set swApp = CreateObject("SldWorks.Application")
    Set pDrawing = swApp.NewDrawing2(13, "", 0, 0.2794, 0.2159)

    ' Get LayerMgr object
    Set pLayerMgr = pDrawing.GetLayerManager

    ' Create a layer
    res = pLayerMgr.AddLayer("myLayer", "Desc", 1227327, swLineStyles_e.swLineCONTINUOUS, swLineWeights_e.swLW_THIN)
    If (res = False) Then
        swApp.SendMsgToUser "Error creating layer."
    End If

    ' Activate the layer
    res = pLayerMgr.SetCurrentLayer("myLayer")
    If (res = False) Then
        swApp.SendMsgToUser "Error activating layer."
    End If

    ' Create some sketch entities on the active layer
    pDrawing.SetAddToDB True
    Set pSketchPoint1 = pDrawing.CreatePoint2(0.1, 0.14, 0)
    Set pSketchPoint2 = pDrawing.CreatePoint2(0.16, 0.14, 0)
    Set pSketchSegment1 = pDrawing.CreateCircle2(0.13, 0.1, 0, 0.2, 0.099, 0)
    Set pSketchSegment2 = pDrawing.CreateLine2(0.13, 0.1, 0, 0.12, 0.1, 0)
    Set pSketchSegment3 = pDrawing.CreateLine2(0.12, 0.1, 0, 0.13, 0.13, 0)
    Set pSketchSegment4 = pDrawing.CreateArc2(0.13, 0.1, 0, 0.09, 0.07, 0, 0.14, 0.051, 0, 1)
    Set pSketchSegment5 = pDrawing.CreateArc2(0.13, 0.1, 0, 0.195, 0.135, 0, 0.065, 0.135, 0, 1)
    pDrawing.SetAddToDB False

    pDrawing.ClearSelection

    Stop
    ' Examine the drawing, then press F5

    ' Change the visual properties of a couple of sketch entities, which
    ' overrides the visual properties of the layer
    pSketchSegment4.Color = 4227327
    pSketchSegment4.Width = swLineWeights_e.swLW_THICK
    pSketchSegment4.Style = swLineStyles_e.swLinePHANTOM
    pSketchSegment5.Color = 0

    ' Repaint to see the visual changes
    pDrawing.GraphicsRedraw2

End Sub
```

Create Sketch Path Example (C#)

This example shows how to create a sketch path.

```
// Preconditions:
// 1. Verify that the specified document template exists.
// 2. Open an Immediate window.
//
// Postconditions:
// 1. Creates a new model document with Sketch1.
// 2. Selects the Sketch1 segments.
// 3. Creates a sketch path.
// 4. Creates a sketch circle.
// 5. Adds a tangent relation between the sketch path and sketch circle.
// 6. Inspect the Immediate window.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace CreateSketchPath_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        SelectionMgr swSelMgr;
        Feature swFeat;
        Sketch swSketch;
        int i;
        bool bRet;
        object[] vSketchPath;
        SketchPath swSketchPath;
        double nLength;
        object[] vConstraint;
        SketchRelation swSkRel;
        object[] vRelation;
        object vSkRel;
        object[] vSketchSeg;
        SketchSegment swSketchSeg;
        SketchManager swSketchMgr;
        bool boolstatus;

        public void Main()
        {
            swModel = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SolidWorks\\SOLIDWORKS 2015\\templates\\Part.prtdot", 0, 0, 0);
            swModel = (ModelDoc2)swApp.ActiveDoc;

            swModel.SketchManager.InsertSketch(true);

            boolstatus = swModel.Extension.SelectByID2("Front Plane", "PLANE", -0.0416217612836357, 0.0270960165695038, 0.00355460240358513, false, 0, 0, 0, 0);

            object skSegment = null;
            skSegment = swModel.SketchManager.CreateLine(-0.055655, 0.037535, 0.0, 0.011848, 0.039924, 0.0);
            skSegment = swModel.SketchManager.CreateLine(0.011848, 0.039924, 0.0, 0.018817, 0.009658, 0.0);
            skSegment = swModel.SketchManager.CreateLine(0.018817, 0.009658, 0.0, 0.05227, 0.008264, 0.0);
            skSegment = swModel.SketchManager.CreateLine(0.05227, 0.008264, 0.0, 0.065809, 0.036414, 0.0);

            swModel.SketchManager.InsertSketch(true);

            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swSketchMgr = swModel.SketchManager;

            // Select the sketch
            bRet = swModel.Extension.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, false, 0, null, 0);

            swFeat = (Feature)swSelMgr.GetSelectedObject6(1, -1);
            swSketch = (Sketch)swFeat.GetSpecificFeature2();

            // Put the sketch in edit mode
            swModel.EditSketch();

            // Get the sketch segments
            vSketchSeg = (object[])swSketch.GetSketchSegments();

            for (i = 0; i <= vSketchSeg.GetUpperBound(0); i++) {
                swSketchSeg = (SketchSegment)vSketchSeg[i];
                swSketchSeg.Select4(true, null);
            }
        }
    }
}
```

```

// Create a chain using the selected sketch segments
bRet = swSketchMgr.MakeSketchChain();
swModel.ClearSelection2(true);

// Get the sketch path
vSketchPath = (object[])swSketch.GetSketchPaths();
swModel.ClearSelection2(true);

// Create a circle
swSketchMgr.CreateCircle(-0.07515069296375, 0.04810565031983, 0, -0.055655, 0.037535, 0);

// Add a tangent relation between sketch path and sketch circle
boolstatus = swModel.Extension.SelectByID("Arc1", "SKETCHSEGMENT", -5.44261072733269E-02, 4.71088420855688E-
02, -3.28513702299429E-03, false, 0, null, 0);
boolstatus = swModel.Extension.SelectByID("Line1", "SKETCHSEGMENT", -4.22450565500339E-02, 3.67651345154678E-
02, -2.68554920844266E-03, true, 0, null, 0);
swModel.SketchAddConstraints("sgTANGENT");

// Print the number of constraints, number of relations,
// path length, number of sketch segments, whether the path was selected,
// type of constraints, and type of relations
for (i = 0; i <= vSketchPath.GetUpperBound(0); i++) {
    swSketchPath = (SketchPath)vSketchPath[i];
    Debug.Print(" Number of constraints: " + swSketchPath.GetConstraintsCount());
    Debug.Print(" Number of relations: " + swSketchPath.GetRelationsCount());
    Debug.Print(" Path length: " + swSketchPath.GetLength());
    Debug.Print(" Number of segments: " + swSketchPath.GetSketchSegmentCount());
    Debug.Print(" Selection result: " + swSketchPath.Select(false, null));

    vConstraint = (object[])swSketchPath.GetConstraints();

    int j = 0;
    j = 0;

    if ((vConstraint != null)) {
        for (j = 0; j <= vConstraint.GetUpperBound(0); j++) {
            Debug.Print(" SketchSegConstraint[" + i + "]: " + vConstraint[j]);
        }
    }

    vRelation = (object[])swSketchPath.GetRelations();

    int k = 0;
    for (k = 0; k <= vRelation.GetUpperBound(0); k++) {
        vSkRel = vRelation[k];
        swSkRel = (SketchRelation)vSkRel;
        Debug.Print(" Relation(" + k + ")");
        Debug.Print(" Type: " + swSkRel.GetRelationType());
    }

    // Get the sketch segments in the sketch path and
    // their lengths
    vSketchSeg = (object[])swSketchPath.GetSketchSegments();

    int l = 0;

    for (l = 0; l <= vSketchSeg.GetUpperBound(0); l++) {
        swSketchSeg = (SketchSegment)vSketchSeg[l];

        // Ignore construction lines
        if (swSketchSeg.ConstructionGeometry == false) {
            // Ignore text
            if ((int)swSketchSegments_e.swSketchTEXT != swSketchSeg.GetType()) {
                nLength = nLength + swSketchSeg.GetLength();
            }
        }
    }

    Debug.Print(" Total path length calculated by segment: " + nLength);
}

swModel.SketchManager.InsertSketch(true);

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>

public SldWorks swApp;
}
}

```

Create Sketch Path Example (VBA)

This example shows how to create a sketch path.

```

'-----
' Preconditions:
' 1. Verify that the specified document template exists.
' 2. Open an Immediate window.
'
' Postconditions:
' 1. Creates a new model document with Sketch1.
' 2. Selects the Sketch1 segments.
' 3. Creates a sketch path.
' 4. Creates a sketch circle.
' 5. Adds a tangent relation between the sketch path and sketch circle.
' 6. Inspect the Immediate window.
'-----
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swSelMgr As SldWorks.SelectionMgr
Dim swFeat As SldWorks.Feature
Dim swSketch As SldWorks.Sketch
Dim i As Long
Dim bRet As Boolean
Dim vSketchPath As Variant
Dim swSketchPath As SldWorks.SketchPath
Dim nLength As Double
Dim vConstraint As Variant
Dim swSkRel As SldWorks.SketchRelation
Dim vRelation As Variant
Dim vSkRel As Variant
Dim vSketchSeg As Variant
Dim swSketchSeg As SldWorks.SketchSegment
Dim swSketchMgr As SldWorks.SketchManager
Dim boolstatus As Boolean

Option Explicit
Sub main()
    Set swApp = Application.SldWorks
    Set swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2015\templates\Part.prtdot", 0, 0, 0)
    Set swModel = swApp.ActiveDoc

    swModel.SketchManager.InsertSketch True
    boolstatus = swModel.Extension.SelectByID2("Front Plane", "PLANE", -4.16217612836357E-02, 2.70960165695038E-02, 3.55460240358513E-03, False, 0, Nothing, 0)

    Dim skSegment As Object
    Set skSegment = swModel.SketchManager.CreateLine(-0.055655, 0.037535, 0#, 0.011848, 0.039924, 0#)
    Set skSegment = swModel.SketchManager.CreateLine(0.011848, 0.039924, 0#, 0.018817, 0.009658, 0#)
    Set skSegment = swModel.SketchManager.CreateLine(0.018817, 0.009658, 0#, 0.05227, 0.008264, 0#)
    Set skSegment = swModel.SketchManager.CreateLine(0.05227, 0.008264, 0#, 0.065809, 0.036414, 0#)

    swModel.SketchManager.InsertSketch True

    Set swSelMgr = swModel.SelectionManager
    Set swSketchMgr = swModel.SketchManager

    ' Select the sketch
    bRet = swModel.Extension.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)

    Set swFeat = swSelMgr.GetSelectedObject6(1, -1)
    Set swSketch = swFeat.GetSpecificFeature2

    ' Put the sketch in edit mode
    swModel.EditSketch

    ' Get the sketch segments
    vSketchSeg = swSketch.GetSketchSegments

    For i = 0 To UBound(vSketchSeg)
        Set swSketchSeg = vSketchSeg(i)
        swSketchSeg.Select4 True, Nothing
    Next

    ' Create a chain using the selected sketch segments
    bRet = swSketchMgr.MakeSketchChain
    swModel.ClearSelection2(True)

    ' Get the sketch path
    vSketchPath = swSketch.GetSketchPaths

    ' Create a circle
    swModel.CreateCircle -0.07515069296375, 0.04810565031983, 0, -0.06525335820896, 0.04189104477612, 0

```

```

' Add a tangent relation between sketch path and sketch circle
boolstatus = swModel.Extension.SelectByID2("Arc1", "SKETCHSEGMENT", -5.44261072733269E-02, 4.71088420855688E-02,
-3.28513702299429E-03, False, 0, Nothing, 0)
boolstatus = swModel.Extension.SelectByID2("Line1", "SKETCHSEGMENT", -4.22450565500339E-02, 3.67651345154678E-02,
-2.68554920844266E-03, True, 0, Nothing, 0)
swModel.SketchAddConstraints ("sgTANGENT")

' Print the number of constraints, number of relations,
' path length, number of sketch segments, whether the path was selected,
' type of constraints, and type of relations
For i = 0 To UBound(vSketchPath)
    Set swSketchPath = vSketchPath(i)
    Debug.Print " Number of constraints: " & swSketchPath.GetConstraintsCount
    Debug.Print " Number of relations: " & swSketchPath.GetRelationsCount
    Debug.Print " Path length: " & swSketchPath.GetLength
    Debug.Print " Number of segments: " & swSketchPath.GetSketchSegmentCount
    Debug.Print " Selection result: " & swSketchPath.Select(False, Nothing)

    vConstraint = swSketchPath.GetConstraints

    Dim j As Integer
    j = 0
    If (Not IsEmpty(vConstraint)) Then
        For j = 0 To UBound(vConstraint)
            Debug.Print " SketchSegConstraint[" & i & "]: " & vConstraint(j)
        Next j
    End If

    vRelation = swSketchPath.GetRelations

    Dim k As Integer
    k = 0
    For Each vSkRel In vRelation
        Set swSkRel = vSkRel
        Debug.Print " Relation[" & k & "]"
        Debug.Print " Type: " & swSkRel.GetRelationType
        k = k + 1
    Next

    ' Get the sketch segments in the sketch path and
    ' their lengths
    vSketchSeg = swSketchPath.GetSketchSegments

    Dim l As Integer
    For l = 0 To UBound(vSketchSeg)
        Set swSketchSeg = vSketchSeg(l)
        ' Ignore construction lines
        If swSketchSeg.ConstructionGeometry = False Then
            ' Ignore text
            If swSketchSeg.TEXT <> swSketchSeg.GetType Then
                nLength = nLength + swSketchSeg.GetLength
            End If
        End If
    Next l

    Debug.Print " Total path length calculated by segment: " & nLength
Next

swModel.SketchManager.InsertSketch True

End Sub

```

Create Sketch Path Example (VB.NET)

This example shows how to create a sketch path.

```

'-----
' Preconditions:
' 1. Verify that the specified document template exists.
' 2. Open an Immediate window.
'
' Postconditions:
' 1. Creates a new model document with Sketch1.
' 2. Selects the Sketch1 segments.
' 3. Creates a sketch path.
' 4. Creates a sketch circle.
' 5. Adds a tangent relation between the sketch path and sketch circle.
' 6. Inspect the Immediate window.
'-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

```

```
Partial Class SolidWorksMacro
```

```

    Dim swModel As ModelDoc2
    Dim swSelMgr As SelectionMgr
    Dim swFeat As Feature
    Dim swSketch As Sketch
    Dim i As Integer
    Dim bRet As Boolean
    Dim vSketchPath As Object
    Dim swSketchPath As SketchPath
    Dim nLength As Double
    Dim vConstraint As Object
    Dim swSkRel As SketchRelation
    Dim vRelation As Object
    Dim vSkRel As Object
    Dim vSketchSeg As Object
    Dim swSketchSeg As SketchSegment
    Dim swSketchMgr As SketchManager
    Dim boolstatus As Boolean

```

```
Sub main()
```

```

    swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2015\templates\Part.prtdot", 0, 0, 0)
    swModel = swApp.ActiveDoc

    swModel.SketchManager.InsertSketch(True)
    boolstatus = swModel.Extension.SelectByID("Front Plane", "PLANE", -0.0416217612836357, 0.0270960165695038, 0.00355460240356

    Dim skSegment As Object
    skSegment = swModel.SketchManager.CreateLine(-0.055655, 0.037535, 0.0#, 0.011848, 0.039924, 0.0#)
    skSegment = swModel.SketchManager.CreateLine(0.011848, 0.039924, 0.0#, 0.018817, 0.009658, 0.0#)
    skSegment = swModel.SketchManager.CreateLine(0.018817, 0.009658, 0.0#, 0.05227, 0.008264, 0.0#)
    skSegment = swModel.SketchManager.CreateLine(0.05227, 0.008264, 0.0#, 0.065809, 0.036414, 0.0#)

    swModel.SketchManager.InsertSketch(True)

    swSelMgr = swModel.SelectionManager
    swSketchMgr = swModel.SketchManager

    ' Select the sketch
    bRet = swModel.Extension.SelectByID("Sketch1", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)

    swFeat = swSelMgr.GetSelectedObject6(1, -1)
    swSketch = swFeat.GetSpecificFeature2

    ' Put the sketch in edit mode
    swModel.EditSketch()

    ' Get the sketch segments
    vSketchSeg = swSketch.GetSketchSegments

    For i = 0 To UBound(vSketchSeg)
        swSketchSeg = vSketchSeg(i)
        swSketchSeg.Select4(True, Nothing)
    Next

    ' Create a chain using the selected sketch segments
    bRet = swSketchMgr.MakeSketchChain
    swModel.ClearSelection2(True)

    ' Get the sketch path
    vSketchPath = swSketch.GetSketchPaths
    swModel.ClearSelection2(True)

```

```

' Create a circle
swSketchMgr.CreateCircle(-0.07515069296375, 0.04810565031983, 0, -0.055655, 0.037535, 0)

' Add a tangent relation between sketch path and sketch circle
boolstatus = swModel.Extension.SelectByID("Arc1", "SKETCHSEGMENT", -0.0544261072733269, 0.0471088420855688, -0.003285137022
boolstatus = swModel.Extension.SelectByID("Line1", "SKETCHSEGMENT", -0.0422450565500339, 0.0367651345154678, -0.00268554926
swModel.SketchAddConstraints("sgTANGENT")

' Print the number of constraints, number of relations,
' path length, number of sketch segments, whether the path was selected,
' type of constraints, and type of relations
For i = 0 To UBound(vSketchPath)

    swSketchPath = vSketchPath(i)
    Debug.Print(" Number of constraints: " & swSketchPath.GetConstraintsCount)
    Debug.Print(" Number of relations: " & swSketchPath.GetRelationsCount)
    Debug.Print(" Path length: " & swSketchPath.GetLength)
    Debug.Print(" Number of segments: " & swSketchPath.GetSketchSegmentCount)
    Debug.Print(" Selection result: " & swSketchPath.Select(False, Nothing))

    vConstraint = swSketchPath.GetConstraints

    Dim j As Integer
    j = 0

    If (Not IsNothing(vConstraint)) Then
        For j = 0 To UBound(vConstraint)
            Debug.Print(" SketchSegConstraint[" & i & "]: " & vConstraint(j))
        Next j
    End If

    vRelation = swSketchPath.GetRelations

    Dim k As Integer
    k = 0

    For Each vSkRel In vRelation
        swSkRel = vSkRel
        Debug.Print(" Relation(" & k & ")")
        Debug.Print(" Type: " & swSkRel.GetRelationType)
        k = k + 1
    Next

    ' Get the sketch segments in the sketch path and
    ' their lengths
    vSketchSeg = swSketchPath.GetSketchSegments

    Dim l As Integer
    For l = 0 To UBound(vSketchSeg)

        swSketchSeg = vSketchSeg(l)

        ' Ignore construction lines
        If swSketchSeg.ConstructionGeometry = False Then
            ' Ignore text
            If swSketchSegments_e.swSketchTEXT <> swSketchSeg.GetType Then
                nLength = nLength + swSketchSeg.GetLength
            End If
        End If
    Next l

    Debug.Print(" Total path length calculated by segment: " & nLength)

Next

swModel.SketchManager.InsertSketch(True)
End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class

```

Create a Sketch Point Example (C#)

This example shows how to create a sketch point.

```
//-----
// Preconditions: Verify that the specified part template exists.
//
// Postconditions:
// 1. Opens a new part document and creates a sketch.
// 2. Creates a sketch point in the sketch.
// 3. Examine the graphics area.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace CreateSketchPoint_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            SketchManager swSkMgr = default(SketchManager);
            int longstatus = 0;
            bool boolstatus = false;

            swApp.ResetUntitledCount(0, 0, 0);
            swModel = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SOLIDWORKS\\SOLIDWORKS 2012\\templates\\Part.prtdot", 0, 0,
0);

            swApp.ActivateDoc2("Part1", false, ref longstatus);
            swModel = (ModelDoc2)swApp.ActiveDoc;

            swSkMgr = swModel.SketchManager;
            swSkMgr.InsertSketch(true);
            boolstatus = swModel.Extension.SelectByID2("Top Plane", "PLANE", -0.0553489443349025, 0.00330468607538553,
0.0269617286188933, false, 0, null, 0);
            swModel.ClearSelection2(true);

            // Check whether document is active
            if (swModel == null)
            {
                swApp.SendMsgToUser2("A part document must be active.", (int) swMessageBoxIcon_e.swMbWarning, (int)
swMessageBoxBtn_e.swMbOk);
                return;
            }

            // Check whether document is a part
            int modelType = 0;
            modelType = swModel.GetType();

            if (modelType != (int)swDocumentTypes_e.swDocPART)
            {
                swApp.SendMsgToUser2("A part document must be active.", (int)swMessageBoxIcon_e.swMbWarning,
(int)swMessageBoxBtn_e.swMbOk);
                return;
            }

            SketchPoint skPoint = default(SketchPoint);
            skPoint = swSkMgr.CreatePoint(-0.127443, 0.042892, 0.0);
            swSkMgr.InsertSketch(true);
        }

        public SldWorks swApp;
    }
}
```

Create a Sketch Point Example (VBA)

This example shows how to create a sketch point.

```
'-----
' Preconditions: Verify that the specified part template exists.
'
' Postconditions:
' 1. Opens a new part document and creates a sketch.
' 2. Creates a sketch point in the sketch.
' 3. Examine the graphics area.
'-----
```

Option Explicit

Sub main()

```
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swSkMgr As SldWorks.SketchManager
Dim longstatus As Long
Dim boolstatus As Boolean
```

```
Set swApp = Application.SldWorks
swApp.ResetUntitledCount 0, 0, 0
Set swModel = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2017\templates\Part.prtdot", 0, 0, 0)
swApp.ActivateDoc2 "Part1", False, longstatus
Set swModel = swApp.ActiveDoc
```

```
Set swSkMgr = swModel.SketchManager
swSkMgr.InsertSketch True
boolstatus = swModel.Extension.SelectByID2("Top Plane", "PLANE", -5.53489443349025E-02, 3.30468607538553E-03,
2.69617286188933E-02, False, 0, Nothing, 0)
swModel.ClearSelection2 True
```

```
' Check whether document is active
If swModel Is Nothing Then
    swApp.SendMsgToUser2 "A part document must be active.", swMbWarning, swMbOk
    Exit Sub
End If
```

```
' Check whether document is a part
Dim modelType As Long
modelType = swModel.GetType
```

```
If modelType <> SwConst.swDocPART Then
    swApp.SendMsgToUser2 "A part document must be active.", swMbWarning, swMbOk
    Exit Sub
End If
```

```
Dim skPoint As SldWorks.SketchPoint
Set skPoint = swSkMgr.CreatePoint(-0.127443, 0.042892, 0#)
swSkMgr.InsertSketch True
```

End Sub